

[dart.dev](#) / [language](#)

Dart Dil Rehberi

Tipler · Desenler · Kontrol akışı · Fonksiyonlar

2. Takım · codemiks

İçindekiler

Her kod örneğinin üstündeki mavi “VS Code’da aç” etiketine ya da kodun kendisine tıklandığında, örneğin `cods/` klasöründeki dosyası VS Code’da açılır. Bunun için PDF, `cods` klasörüyle aynı klasörde durmalıdır.

Tipler

- [Yerleşik tipler](#)
- [Record'lar](#)
- [Koleksiyonlar](#)
- [Generic'ler](#)
- [Typedef'ler](#)
- [Dart tip sistemi](#)

Desenler

- [Desenler](#)
- [Desen tipleri](#)

Kontrol akışı

- [Döngüler](#)
- [Dallanmalar](#)
- [Hata yönetimi](#)

Fonksiyonlar

- [Fonksiyonlar](#)

Yerleşik tipler

Dart'ın desteklediği tipler hakkında bilgi.

Dart dili aşağıdakiler için özel destek sunar:

- **Sayılar** (`int`, `double`)
- **String'ler** (`String`)
- **Boolean'lar** (`bool`)
- **Record'lar** (`(value1, value2)`)
- **Fonksiyonlar** (`Function`)
- **Listeler** (`List`, *dizi (array)* olarak da bilinir)
- **Set'ler** (`Set`)
- **Map'ler** (`Map`)
- **Rune'lar** (`Runes`; çoğunlukla `characters` API'si ile değiştirilir)
- **Sembol'ler** (`Symbol`)
- `null` değeri (`Null`)

Bu destek, literal'ler kullanarak nesne oluşturabilmeyi de kapsar. Örneğin `'this is a string'` bir string literal'idir, `true` ise bir boolean literal'idir.

Dart'ta her değişken bir nesneye (bir *sınıfın (class)* örneğine) başvurduğu için değişkenleri genellikle *yapıcılar (constructor)* ile başlatabilirsiniz. Yerleşik tiplerin bazılarının kendi yapıcıları vardır. Örneğin bir map oluşturmak için `Map()` yapıcısını kullanabilirsiniz.

Bazı başka tiplerin de Dart dilinde özel rolleri vardır:

- **Object**: `Null` dışındaki tüm Dart sınıflarının üst sınıfı (superclass).
- **Enum**: Tüm enum'ların üst sınıfı.
- **Future** ve **Stream**: **Asenkron programlamada** kullanılır.
- **Iterable**: **for-in döngülerinde** ve senkron **üreteç (generator) fonksiyonlarında** kullanılır.
- **Never**: Bir ifadenin değerlendirilmesinin asla başarıyla tamamlanamayacağını belirtir. Çoğunlukla her zaman istisna fırlatan fonksiyonlarda kullanılır.
- **dynamic**: Statik kontrolü kapatmak istediğinizi belirtir. Genellikle bunun yerine `Object` ya da `Object?` kullanmalısınız.
- **void**: Bir değerin hiçbir zaman kullanılmadığını belirtir. Çoğunlukla dönüş tipi olarak kullanılır.

`Object`, `Object?`, `Null` ve `Never` sınıflarının sınıf hiyerarşisinde özel rolleri vardır. Bu roller hakkında bilgi için [Understanding null safety](#) sayfasına bakın.

Sayılar

Dart'ta sayılar iki çeşittir:

`int`

Platforma bağlı olarak 64 biti aşmayan tam sayı değerleri. Yerel (native) platformlarda değerler -2^{63} ile $2^{63} - 1$ arasında olabilir. Web'de tam sayılar JavaScript sayıları (kesirli kısmı olmayan 64 bit kayan noktalı değerler) olarak temsil edilir ve -2^{53} ile $2^{53} - 1$ arasında olabilir.

`double`

IEEE 754 standardında tanımlandığı şekliyle 64 bit (çift duyarlıklı) kayan noktalı sayılar.

`int` ve `double`, `num` tipinin alt tipleridir. `num` tipi `+`, `-`, `/` ve `*` gibi temel operatörleri içerir; `abs()`, `ceil()` ve `floor()` gibi metotlar da burada bulunur. (`>>` gibi bit düzeyi operatörler `int` sınıfında tanımlıdır.) Aradığınız şey `num` ve alt tiplerinde yoksa `dart:math` kütüphanesinde olabilir.

Tam sayılar ondalık noktası olmayan sayılardır. Tam sayı literal'i tanımlamaya birkaç örnek:

[VS Code'da aç](#) `cods/1_types/1_built_in_types/01_integer_literals.dart`

dart

```
var x = 1;
var hex = 0xDEADBEEF;
```

Bir sayı ondalık nokta içeriyorsa double'dır. Double literal'i tanımlamaya birkaç örnek:

[VS Code'da aç](#) `cods/1_types/1_built_in_types/02_double_literals.dart`

dart

```
var y = 1.1;
var exponents = 1.42e5;
```

Bir değişkeni num olarak da tanımlayabilirsiniz. Bu durumda değişken hem tam sayı hem de double değer alabilir.

[VS Code'da aç](#) [cods/1_types/1_built_in_types/03_declare_num.dart](#)

dart

```
num x = 1; // x hem int hem de double değer alabilir
x += 2.5;
```

Tam sayı literal'leri gerektiğinde otomatik olarak double'a çevrilir:

[VS Code'da aç](#) [cods/1_types/1_built_in_types/04_int_to_double.dart](#)

dart

```
double z = 1; // double z = 1.0 ile aynıdır.
```

Bir string'i sayıya çevirmek ya da tersini yapmak şöyle olur:

[VS Code'da aç](#) [cods/1_types/1_built_in_types/05_number_conversion.dart](#)

dart

```
// String -> int
var one = int.parse('1');
assert(one == 1);

// String -> double
var onePointOne = double.parse('1.1');
assert(onePointOne == 1.1);

// int -> String
String oneAsString = 1.toString();
assert(oneAsString == '1');

// double -> String
String piAsString = 3.14159.toStringAsFixed(2);
assert(piAsString == '3.14');
```

`int` tipi geleneksel bit kaydırma (`<<`, `>>`, `>>>`), tümleyen (`~`), AND (`&`), OR (`|`) ve XOR (`^`) operatörlerini tanımlar. Bunlar bit alanlarındaki bayrakları değiştirmek ve maskelemek için kullanışlıdır. Örneğin:

[VS Code'da aç](#) [cods/1_types/1_built_in_types/06_bit_shifting.dart](#)

dart

```
assert((3 << 1) == 6); // 0011 << 1 == 0110
assert((3 | 4) == 7); // 0011 | 0100 == 0111
assert((3 & 4) == 0); // 0011 & 0100 == 0000
```

Daha fazla örnek için [bit düzeyi ve kaydırma operatörleri](#) bölümüne bakın.

Sayı literal'leri derleme zamanı sabitleridir (compile-time constant). Birçok aritmetik ifade de, işlenenleri sayı sonucu veren derleme zamanı sabitleri olduğu sürece derleme zamanı sabitidir.

[VS Code'da aç](#) [cods/1_types/1_built_in_types/07_const_num.dart](#)

dart

```
const msPerSecond = 1000;
const secondsUntilRetry = 5;
const msUntilRetry = secondsUntilRetry * msPerSecond;
```

Daha fazla bilgi için [Numbers in Dart](#) sayfasına bakın.

Uzun sayı literal'lerini daha okunur hale getirmek için bir ya da daha fazla alt çizgiyi (`_`) basamak ayırıcı olarak kullanabilirsiniz. Birden fazla basamak ayırıcı, daha üst düzey gruplamaya izin verir.

[VS Code'da aç](#) `cods/1_types/1_built_in_types/08_digit_separators.dart`

dart

```
var n1 = 1_000_000;
var n2 = 0.000_000_000_01;
var n3 = 0x00_14_22_01_23_45; // MAC adresi
var n4 = 555_123_4567; // ABD telefon numarası
var n5 = 100__000_000__000_000; // yüz milyon milyon!
```

↑ Sürüm notu

Basamak ayırıcıları kullanmak en az 3.6 [dil sürümü](#) gerektirir.

String'ler

Bir Dart string'i (`String` nesnesi) UTF-16 kod birimlerinden oluşan bir dizi tutar. String oluşturmak için tek ya da çift tırnak kullanabilirsiniz:

[VS Code'da aç](#) `cods/1_types/1_built_in_types/09_quoting.dart`

dart

```
var s1 = 'Single quotes work well for string literals.';
var s2 = "Double quotes work just as well.";
var s3 = 'It\'s easy to escape the string delimiter.';
var s4 = "It's even easier to use the other delimiter.";
```

Bir ifadenin değerini string içine `{{ ifade }}` şeklinde yerleştirebilirsiniz. İfade bir tanımlayıcıysa `{}}` kısmını atlayabilirsiniz. Bir nesneye karşılık gelen string'i elde etmek için Dart, nesnenin `toString()` metodunu çağırır.

[VS Code'da aç](#) `cods/1_types/1_built_in_types/10_string_interpolation.dart`

dart

```
var s = 'string interpolation';

assert(
  'Dart has $s, which is very handy.' ==
    'Dart has string interpolation, '
      'which is very handy.',
);
assert(
  'That deserves all caps. '
    '$s.toUpperCase() is very handy!' ==
    'That deserves all caps. '
      'STRING INTERPOLATION is very handy!',
);
```

ⓘ Not

`==` operatörü iki nesnenin eşdeğer olup olmadığını test eder. İki string aynı kod birimi dizisini içeriyorsa eşdeğerdir.

String'leri yan yana yazılmış string literal'leriyle ya da `+` operatörüyle birleştirebilirsiniz:

[VS Code'da aç](#) `cods/1_types/1_built_in_types/11_adjacent_string_literals.dart`

dart

```
var s1 =
  'String '
  'concatenation'
  " works even over line breaks.";
assert(
  s1 ==
    'String concatenation works even over '
    'line breaks.',
);

var s2 = 'The + operator ' + 'works, as well.';
assert(s2 == 'The + operator works, as well.');
```

Çok satırlı bir string oluşturmak için tek ya da çift tırnakla üçlü tırnak kullanın:

[VS Code'da aç](#) `cods/1_types/1_built_in_types/12_triple_quotes.dart`

dart

```
var s1 = '''
You can create
multi-line strings like this one.
''';

var s2 = """This is also a
multi-line string.""";
```

Başına `r` ekleyerek "ham" (raw) bir string oluşturabilirsiniz:

[VS Code'da aç](#) `cods/1_types/1_built_in_types/13_raw_strings.dart`

dart

```
var s = r'In a raw string, not even \n gets special treatment.';
```

Bir string içinde Unicode karakterlerin nasıl ifade edileceği için [Rune'lar ve grafem kümeleri](#) bölümüne bakın.

String literal'leri, içine yerleştirilen her ifade null ya da sayı, string veya boolean değer veren bir derleme zamanı sabiti olduğu sürece derleme zamanı sabitidir.

[VS Code'da aç](#) `cods/1_types/1_built_in_types/14_string_literals.dart`

dart

```
// Bunlar const bir string içinde çalışır.
const aConstNum = 0;
const aConstBool = true;
const aConstString = 'a constant string';

// Bunlar const bir string içinde ÇALIŞMAZ.
var aNum = 0;
var aBool = true;
var aString = 'a string';
const aConstList = [1, 2, 3];

const validConstString = '$aConstNum $aConstBool $aConstString';
// const invalidConstString = '$aNum $aBool $aString $aConstList';
```

String kullanımı hakkında daha fazla bilgi için [Strings and regular expressions](#) sayfasına göz atın.

Boolean'lar

Dart'ta boolean değerleri temsil etmek için `bool` adlı bir tip vardır. Yalnızca iki nesne bool tipindedir: `true` ve `false` boolean literal'leri. İkisi de derleme zamanı sabitidir.

Dart'ın tip güvenliği nedeniyle `if (booleanOlmayanDeger)` ya da `assert (booleanOlmayanDeger)` gibi kodlar yazamazsınız. Bunun yerine değerleri şu şekilde açıkça kontrol edin:

[VS Code'da aç](#) cods/1_types/1_built_in_types/15_no_truthy.dart

dart

```
// Boş string kontrolü.
var fullName = '';
assert(fullName.isEmpty);

// Sıfır kontrolü.
var hitPoints = 0;
assert(hitPoints == 0);

// null kontrolü.
var unicorn = null;
assert(unicorn == null);

// NaN kontrolü.
var iMeantToDoThis = 0 / 0;
assert(iMeantToDoThis.isNaN);
```

Rune'lar ve grafem kümeleri

Dart'ta **rune'lar** bir string'in Unicode kod noktalarını sunar. Kullanıcının algıladığı karakterleri, yani **Unicode (genişletilmiş) grafem kümelerini (grapheme cluster)** görüntülemek ya da değiştirmek için **characters paketini** kullanabilirsiniz.

Unicode, dünyadaki tüm yazı sistemlerinde kullanılan her harf, rakam ve sembol için benzersiz bir sayısal değer tanımlar. Dart string'i UTF-16 kod birimlerinden oluşan bir dizi olduğundan, bir string içinde Unicode kod noktalarını ifade etmek özel bir sözdizimi gerektirir. Bir Unicode kod noktasını ifade etmenin olağan yolu `\uXXXX` biçimidir; burada XXXX 4 basamaklı onaltılık (hex) bir değerdir. Örneğin kalp karakteri (♥) `\u2665` şeklinde yazılır. 4'ten fazla ya da az onaltılık basamak belirtmek için değeri süslü parantez içine alın. Örneğin gülen emoji (😄) `\u{1f606}` şeklinde yazılır.

Tek tek Unicode karakterleri okumanız ya da yazmanız gerekiyorsa characters paketinin String üzerinde tanımladığı `characters` getter'ını kullanın. Dönen `Characters` nesnesi, string'in grafem kümelerinden oluşan bir dizi halidir. characters API'sinin kullanımına bir örnek:

[VS Code'da aç](#) cods/1_types/1_built_in_types/16_ornek.dart

dart

```
import 'package:characters/characters.dart';

void main() {
  var hi = 'Hi 🇩🇰';
  print(hi);
  print('The end of the string: ${hi.substring(hi.length - 1)}');
  print('The last character: ${hi.characters.last}');
}
```

Çıktı, ortamınıza bağlı olarak aşağı yukarı şöyle görünür:

```
$ dart run bin/main.dart
Hi 🇩🇰
The end of the string: ???
The last character: 🇩🇰
```

characters paketiyle string'leri işleme hakkında ayrıntılar için paketin [örneğine](#) ve [API referansına](#) bakın.

Sembol'ler

Bir `Symbol` nesnesi, bir Dart programında tanımlanmış bir operatörü ya da tanımlayıcıyı temsil eder. Sembol kullanmanız hiç gerekemeyebilir; ancak tanımlayıcılara isimleriyle başvuran API'ler için çok değerlidirler, çünkü küçültme (minification) tanımlayıcı isimlerini değiştirir ama tanımlayıcı sembollerini değiştirmez.

Bir tanımlayıcının sembolünü almak için bir sembol literal'i kullanın. Bu, tanımlayıcının önüne `#` koymaktan ibarettir:

```
#radix  
#bar
```

Sembol literal'leri derleme zamanı sabitleridir.

Record'lar

Dart'taki record veri yapısının özeti.

↑ Sürüm notu

Record'lar en az 3.0 [dil sürümü](#) gerektirir.

Record'lar anonim, değiştirilemez (immutable), birleşik bir tiptir. Diğer [koleksiyon tipleri](#) gibi birden fazla nesneyi tek bir nesnede toplamanızı sağlarlar. Diğer koleksiyon tiplerinden farklı olarak record'lar sabit boyutlu, heterojen ve tiplidir.

Record'lar gerçek değerlerdir; onları değişkenlerde saklayabilir, iç içe kullanabilir, fonksiyonlara geçirip fonksiyonlardan döndürebilir ve list, map, set gibi veri yapılarında saklayabilirsiniz.

Record sözdizimi

Record ifadeleri, parantez içine alınmış, virgülle ayrılmış isimli ya da konumsal (positional) alan listeleridir:

[VS Code'da aç](#) `cods/1_types/2_records/01_record_syntax.dart`

```
var record = ('first', a: 2, b: true, 'last');
```

dart

Record tip belirtimleri (type annotation), parantez içine alınmış, virgülle ayrılmış tip listeleridir. Record tip belirtimlerini dönüş tiplerini ve parametre tiplerini tanımlamak için kullanabilirsiniz. Örneğin aşağıdaki `(int, int)` ifadeleri record tip belirtimleridir:

[VS Code'da aç](#) `cods/1_types/2_records/02_record_type_annotation.dart`

```
(int, int) swap((int, int) record) {
  var (a, b) = record;
  return (b, a);
}
```

dart

Record ifadelerindeki ve tip belirtimlerindeki alanlar, fonksiyonlardaki [parametre ve argümanların](#) işleyişini yansıtır. Konumsal alanlar doğrudan parantezin içine yazılır:

[VS Code'da aç](#) `cods/1_types/2_records/03_record_type_declaration.dart`

```
// Değişken tanımında record tipi belirtimi:
(String, int) record;

// Bir record ifadesiyle başlat:
record = ('A string', 123);
```

dart

Bir record tip belirtiminde isimli alanlar, tüm konumsal alanlardan sonra, süslü parantezle ayrılmış tip-isim çiftlerinden oluşan bir bölüme yazılır. Bir record ifadesinde ise isimler her alan değerinden önce, ardından iki nokta gelecek şekilde yazılır:

[VS Code'da aç](#) `cods/1_types/2_records/04_record_type_named_declaration.dart`

```
// Değişken tanımında record tipi belirtimi:
({int a, bool b}) record;

// Bir record ifadesiyle başlat:
record = (a: 123, b: true);
```

dart

Bir record tipindeki isimli alanların isimleri, [record'un tip tanımının](#), yani *şeklinin (shape)* bir parçasıdır. İsimli alanlarının isimleri farklı olan iki record farklı tiptedir:

[VS Code'da aç](#) [cods/1_types/2_records/05_record_type_mismatched_names.dart](#)

dart

```
{int a, int b} recordAB = (a: 1, b: 2);
{int x, int y} recordXY = (x: 3, y: 4);

// Derleme hatası! Bu record'lar aynı tipte değil.
// recordAB = recordXY;
```

Bir record tip belirtiminde *konumsal* alanlara da isim verebilirsiniz; ancak bu isimler yalnızca belgeleme amaçlıdır ve record'un tipini etkilemez:

[VS Code'da aç](#) [cods/1_types/2_records/06_record_type_matched_names.dart](#)

dart

```
(int a, int b) recordAB = (1, 2);
(int x, int y) recordXY = (3, 4);

recordAB = recordXY; // Sorun yok.
```

Bu, bir [fonksiyon tanımında ya da fonksiyon typedef'inde](#) konumsal parametrelerin isimleri olabilmesine, ama bu isimlerin fonksiyonun imzasını etkilememesine benzer.

Daha fazla bilgi ve örnek için [Record tipleri](#) ve [Record eşitliği](#) bölümlerine göz atın.

Record alanları

Record alanlarına yerleşik getter'lar üzerinden erişilir. Record'lar değiştirilemez olduğu için alanların setter'ı yoktur.

İsimli alanlar aynı isimde getter'lar sunar. Konumsal alanlar ise isimli alanlar atlanarak `$<konum>` isimli getter'lar sunar:

[VS Code'da aç](#) [cods/1_types/2_records/07_record_getters.dart](#)

dart

```
var record = ('first', a: 2, b: true, 'last');

print(record.$1); // 'first' yazdırır
print(record.a); // 2 yazdırır
print(record.b); // true yazdırır
print(record.$2); // 'last' yazdırır
```

Record alanlarına erişimi daha da kolaylaştırmak için [Desenler](#) sayfasına göz atın.

Record tipleri

Tek tek record tipleri için bir tip tanımı yoktur. Record'lar alanlarının tiplerine göre yapısal olarak tiplendirilir. Bir record'un *şekli* (alanlarının kümesi, alanların tipleri ve varsa isimleri) record'un tipini benzersiz şekilde belirler.

Bir record'daki her alanın kendi tipi vardır. Aynı record içinde alan tipleri farklı olabilir. Tip sistemi, record'dan hangi noktada erişilirse erişilsin her alanın tipini bilir:

[VS Code'da aç](#) [cods/1_types/2_records/08_record_getters_two.dart](#)

dart

```
(num, Object) pair = (42, 'a');

var first = pair.$1; // Statik tip 'num', çalışma zamanı tipi 'int'.
var second = pair.$2; // Statik tip 'Object', çalışma zamanı tipi 'String'.
```

Aynı alan kümesine sahip record'lar oluşturan, birbiriyle ilgisiz iki kütüphane düşünün. Kütüphaneler birbirine bağlı olmasa da tip sistemi bu record'ların aynı tipte olduğunu anlar.

İpucu

Bir record şekli için benzersiz bir tip tanımlayamasanız da okunabilirlik ve yeniden kullanım için tip takma adları (type alias) oluşturabilirsiniz. Bunu nasıl ve ne zaman yapacağınızı öğrenmek için [Record'lar ve typedef'ler](#) bölümüne göz atın.

Record eşitliği

İki record aynı *şekle* (alan kümesine) sahipse ve karşılık gelen alanları aynı değerleri taşıyorsa eşittir. İsimli alanların *sırası* record'un şeklinin bir parçası olmadığından, isimli alanların sırası eşitliği etkilemez.

Örneğin:

[VS Code'da aç](#) `cods/1_types/2_records/09_record_shape.dart`

dart

```
(int x, int y, int z) point = (1, 2, 3);
(int r, int g, int b) color = (1, 2, 3);

print(point == color); // 'true' yazdırır.
```

[VS Code'da aç](#) `cods/1_types/2_records/10_record_shape_mismatch.dart`

dart

```
{int x, int y, int z} point = (x: 1, y: 2, z: 3);
{int r, int g, int b} color = (r: 1, g: 2, b: 3);

print(point == color); // 'false' yazdırır. Lint: İlgisiz tipler üzerinde eşitlik.
```

Record'lar, alanlarının yapısına göre `hashCode` ve `==` metotlarını otomatik olarak tanımlar.

Birden çok değer döndürme

Record'lar, fonksiyonların bir arada paketlenmiş birden çok değer döndürmesini sağlar. Dönen record'daki değerleri almak için [desen eşleştirme](#) ile değerleri yerel değişkenlere [ayırıştırın \(destructure\)](#).

[VS Code'da aç](#) `cods/1_types/2_records/11_record_multiple_returns.dart`

dart

```
// Bir record içinde birden çok değer döndürür:
(String name, int age) userInfo(Map<String, dynamic> json) {
  return (json['name'] as String, json['age'] as int);
}

final json = <String, dynamic>{'name': 'Dash', 'age': 10, 'color': 'blue'};

// Konumsal alanlı bir record deseniyle ayırıştırır:
var (name, age) = userInfo(json);

/* Şuna denktir:
var info = userInfo(json);
var name = info.$1;
var age = info.$2;
*/
```

Bir record'u [isimli alanlarını](#) kullanarak, iki nokta `:` sözdizimiyle de ayırıştırabilirsiniz. Bu konuyu [Desen tipleri](#) sayfasında daha ayrıntılı okuyabilirsiniz:

[VS Code'da aç](#) `cods/1_types/2_records/12_record_name_destructure.dart`

dart

```
{String name, int age} userInfo(Map<String, dynamic> json)
// ...
// İsimli alanlı bir record deseniyle ayırıştırır:
final (:name, :age) = userInfo(json);
```

Bir fonksiyondan record olmadan da birden çok değer döndürebilirsiniz, ancak diğer yöntemlerin dezavantajları vardır. Örneğin bir sınıf oluşturmak çok daha uzun sürer; `List` ya da `Map` gibi diğer koleksiyon tiplerini kullanmak ise tip güvenliğini kaybettirir.

Not

Record'ların birden çok değer döndürme ve heterojen tip özellikleri, farklı tipteki future'ların paralel çalıştırılmasını mümkün kılar. Bunu [dart:async dokümantasyonunda](#) okuyabilirsiniz.

Basit veri yapıları olarak record'lar

Record'lar yalnızca veri tutar. İhtiyacınız sadece buysa, yeni bir sınıf tanımlamaya gerek kalmadan hemen kullanıma hazırlarlar ve kullanımları kolaydır. Hepsi aynı şekle sahip veri demetlerinden (tuple) oluşan basit bir liste için *record listesi* en doğrudan temsildir.

Örneğin şu "buton tanımları" listesini ele alalım:

[VS Code'da aç](#) cods/1_types/2_records/13_ornek.dart

```
final buttons = [
  (
    label: "Button I",
    icon: const Icon(Icons.upload_file),
    onPressed: () => print("Action -> Button I"),
  ),
  (
    label: "Button II",
    icon: const Icon(Icons.info),
    onPressed: () => print("Action -> Button II"),
  )
];
```

Bu kod, ek bir tanıma ihtiyaç duymadan doğrudan yazılabilir.

Record'lar ve typedef'ler

Record tipinin kendisine bir isim vermek için [typedef](#) kullanmayı seçebilir ve tam record tipini yazmak yerine bu ismi kullanabilirsiniz. Bu yöntem, listedeki mevcut girdilerin hiçbirinde null değer olmasa bile bazı alanların null olabileceğini (?) belirtmenize olanak tanır.

[VS Code'da aç](#) cods/1_types/2_records/14_ornek.dart

```
typedef ButtonItem = ({String label, Icon icon, void Function()? onPressed});
final List<ButtonItem> buttons = [
  // ...
];
```

Record tipleri yapısal tipler olduğundan, `ButtonItem` gibi bir isim vermek yalnızca yapısal tipe başvurmayı kolaylaştıran bir takma ad oluşturur: `({String label, Icon icon, void Function()? onPressed})`.

Tüm kodunuzun record tipine takma adıyla başvurması, ileride record'un uygulamasını her başvuruyu güncellemeye gerek kalmadan değiştirmeyi kolaylaştırır.

Kod, verilen buton tanımlarıyla basit sınıf örnekleriyle çalışır gibi çalışabilir:

[VS Code'da aç](#) cods/1_types/2_records/15_ornek.dart

```
List<Container> widget = [
  for (var button in buttons)
    Container(
      margin: const EdgeInsets.all(4.0),
      child: OutlinedButton.icon(
        onPressed: button.onPressed,
        icon: button.icon,
        label: Text(button.label),
      ),
    ),
];
```

Hatta ileride metod eklemek için record tipini bir sınıf tipine çevirmeye bile karar verebilirsiniz:

[VS Code'da aç](#) `cods/1_types/2_records/16_ornek.dart`

dart

```
class ButtonItem {
  final String label;
  final Icon icon;
  final void Function()? onPressed;
  ButtonItem({required this.label, required this.icon, this.onPressed});
  bool get hasOnPressed => onPressed != null;
}
```

Ya da bir extension type'a:

[VS Code'da aç](#) `cods/1_types/2_records/17_ornek.dart`

dart

```
extension type ButtonItem._(({String label, Icon icon, void Function()? onPressed}) _) {
  String get label => _.label;
  Icon get icon => _.icon;
  void Function()? get onPressed => _.onPressed;
  ButtonItem({required String label, required Icon icon, void Function()? onPressed})
    : this._((label: label, icon: icon, onPressed: onPressed));
  bool get hasOnPressed => _.onPressed != null;
}
```

Ve ardından buton tanımları listesini o tipin yapıcılarını kullanarak oluşturabilirsiniz:

[VS Code'da aç](#) `cods/1_types/2_records/18_ornek.dart`

dart

```
final List<ButtonItem> buttons = [
  ButtonItem(
    label: "Button I",
    icon: const Icon(Icons.upload_file),
    onPressed: () => print("Action -> Button I"),
  ),
  ButtonItem(
    label: "Button II",
    icon: const Icon(Icons.info),
    onPressed: () => print("Action -> Button II"),
  )
];
```

Bunların hepsi, yine o listeyi kullanan kodu değiştirmeye gerek kalmadan yapılır.

Bir tipi değiştirmek, onu kullanan kodun varsayımlarda bulunmamaya çok dikkat etmesini gerektirir. Bir tip takma adı, onu başvuru olarak kullanan kod için takma adı verilen değer bir record olduğuna dair hiçbir koruma ya da garanti sunmaz. Extension type'lar da çok az koruma sunar. Tam soyutlama ve kapsüllemeyi yalnızca bir sınıf sağlayabilir.

Koleksiyonlar

Dart'taki farklı koleksiyon tiplerinin özeti.

Dart'ta list, set ve map [koleksiyonları](#) için yerleşik destek vardır. Koleksiyonların içerdiği tipleri ayarlamak hakkında daha fazla bilgi için [Generic'ler](#) sayfasına göz atın.

Listeler

Neredeyse her programlama dilinde belki de en yaygın koleksiyon *dizi (array)*, yani sıralı nesne grubudur. Dart'ta diziler [List](#) nesneleridir, bu yüzden çoğu kişi onlara kısaca *liste* der.

Dart list literal'leri, köşeli parantez (`[]`) içine alınmış, virgülle ayrılmış elemanlardan oluşur. Her eleman genellikle bir ifadedir. İşte basit bir Dart listesi:

[VS Code'da aç](#) `cods/1_types/3_collections/01_list_literal.dart`

```
var list = [1, 2, 3];
```

dart

Not

Dart, `list` 'in tipini `List<int>` olarak çıkarır. Bu listeye tam sayı olmayan nesneler eklemeye çalışırsanız analizci ya da çalışma zamanı hata verir. Daha fazla bilgi için [tip çıkarımı](#) konusunu okuyun.

Bir Dart koleksiyon literal'inde son elemandan sonra virgül koyabilirsiniz. Bu *sondaki virgül (trailing comma)* koleksiyonu etkilemez, ancak kopyala-yapıştır hatalarını önlemeye yardımcı olabilir.

[VS Code'da aç](#) `cods/1_types/3_collections/02_trailing_commas.dart`

```
var list = ['Car', 'Boat', 'Plane',,];
```

dart

Listelerde indeksleme sıfırdan başlar: 0 ilk elemanın indeksi, `list.length - 1` ise son elemanın indeksidir. Bir listenin uzunluğunu `.length` özelliğiyle alabilir, elemanlarına da indis operatörüyle (`[]`) erişebilirsiniz:

[VS Code'da aç](#) `cods/1_types/3_collections/03_list_indexing.dart`

```
var list = [1, 2, 3];
assert(list.length == 3);
assert(list[1] == 2);

list[1] = 1;
assert(list[1] == 1);
```

dart

Derleme zamanı sabiti olan bir liste oluşturmak için list literal'inin önüne `const` ekleyin:

[VS Code'da aç](#) `cods/1_types/3_collections/04_const_list.dart`

```
var constantList = const [1, 2, 3];
// constantList[1] = 1; // Bu satır hataya yol açar.
```

dart

Listeler hakkında daha fazla bilgi için [dart:core](#) dokümantasyonunun Lists bölümüne bakın.

Set'ler

Dart'ta set, benzersiz elemanlardan oluşan sırasız bir koleksiyondur. Dart'ta set desteği, set literal'leri ve [Set](#) tipiyle sağlanır.

İşte bir set literal'i ile oluşturulmuş basit bir Dart set'i:

[VS Code'da aç](#) `cods/1_types/3_collections/05_set_literal.dart`

```
var halogens = {'fluorine', 'chlorine', 'bromine', 'iodine', 'astatine'};
```

dart

❗ Not

Dart, `halogens`'in tipini `Set<String>` olarak çıkarır. Set'e yanlış tipte bir eleman eklemeye çalışırsanız analizci ya da çalışma zamanı hata verir. Daha fazla bilgi için [tip çıkarımı](#) konusunu okuyun.

Boş bir set oluşturmak için önünde tip argümanı olan `{}` kullanın ya da `{}`'i `Set` tipinde bir değişkene atayın:

[VS Code'da aç](#) `cods/1_types/3_collections/06_set_vs_map.dart`

dart

```
var names = <String>{};
// Set<String> names = {}; // Bu da çalışır.
// var names = {}; // Set değil, map oluşturur.
```

❗ Set mi, map mi?

Map literal'lerinin sözdizimi set literal'lerinkine benzer. Map literal'leri daha önce geldiği için `{}` varsayılan olarak `Map` tipindedir. `{}` üzerindeki ya da atandığı değişkendeki tip belirtimini unutursanız Dart `Map<dynamic, dynamic>` tipinde bir nesne oluşturur.

Var olan bir set'e `add()` ya da `addAll()` metotlarıyla eleman ekleyin:

[VS Code'da aç](#) `cods/1_types/3_collections/07_set_add_items.dart`

dart

```
var elements = <String>{};
elements.add('fluorine');
elements.addAll(halogens);
```

Set'teki eleman sayısını almak için `.length` kullanın:

[VS Code'da aç](#) `cods/1_types/3_collections/08_set_length.dart`

dart

```
var elements = <String>{};
elements.add('fluorine');
elements.addAll(halogens);
assert(elements.length == 5);
```

Derleme zamanı sabiti olan bir set oluşturmak için set literal'inin önüne `const` ekleyin:

[VS Code'da aç](#) `cods/1_types/3_collections/09_const_set.dart`

dart

```
final constantSet = const {
  'fluorine',
  'chlorine',
  'bromine',
  'iodine',
  'astatine',
};
// constantSet.add('helium'); // Bu satır hataya yol açar.
```

Set'ler hakkında daha fazla bilgi için [dart:core](#) dokümantasyonunun Sets bölümüne bakın.

Map'ler

Bir map'te her eleman bir anahtar-değer (key-value) çiftidir. Bir çiftteki her anahtar bir değerle ilişkilidir; hem anahtarlar hem de değerler herhangi bir tipte nesne olabilir. Aynı değer birden çok farklı anahtarla ilişkilendirilebilse de her anahtar yalnızca bir kez bulunabilir. Dart'ta map desteği, map literal'leri ve `Map` tipiyle sağlanır.

İşte map literal'leriyle oluşturulmuş birkaç basit Dart map'i:

VS Code'da aç [cods/1_types/3_collections/10_map_literal.dart](#)

dart

```
var gifts = {  
  // Anahtar: Değer  
  'first': 'partridge',  
  'second': 'turtledoves',  
  'fifth': 'golden rings',  
};  
  
var nobleGases = {2: 'helium', 10: 'neon', 18: 'argon'};
```

Not

Dart, `gifts`'in tipini `Map<String, String>`, `nobleGases`'in tipini ise `Map<int, String>` olarak çıkarır. İki map'ten birine yanlış tipte bir değer eklemeye çalışırsanız analizci ya da çalışma zamanı hata verir. Daha fazla bilgi için [tip çıkarımı](#) konusunu okuyun.

Aynı nesneleri bir Map yapıcısıyla da oluşturabilirsiniz:

VS Code'da aç [cods/1_types/3_collections/11_map_constructor.dart](#)

dart

```
var gifts = Map<String, String>();  
gifts['first'] = 'partridge';  
gifts['second'] = 'turtledoves';  
gifts['fifth'] = 'golden rings';  
  
var nobleGases = Map<int, String>();  
nobleGases[2] = 'helium';  
nobleGases[10] = 'neon';  
nobleGases[18] = 'argon';
```

Not

C# ya da Java gibi bir dilden geliyorsanız sadece `Map()` yerine `new Map()` görmeyi bekleyebilirsiniz. Dart'ta `new` anahtar kelimesi isteğe bağlıdır. Ayrıntılar için [Using constructors](#) sayfasına bakın.

Var olan bir map'e indis atama operatörüyle (`[]=`) yeni bir anahtar-değer çifti ekleyin:

VS Code'da aç [cods/1_types/3_collections/12_map_add_item.dart](#)

dart

```
var gifts = {'first': 'partridge'};  
gifts['fourth'] = 'calling birds'; // Bir anahtar-değer çifti ekle
```

Bir map'ten indis operatörüyle (`[]`) değer alın:

VS Code'da aç [cods/1_types/3_collections/13_map_retrieve_item.dart](#)

dart

```
var gifts = {'first': 'partridge'};  
assert(gifts['first'] == 'partridge');
```

Map'te olmayan bir anahtarı ararsanız karşılığında `null` alırsınız:

VS Code'da aç [cods/1_types/3_collections/14_map_missing_key.dart](#)

dart

```
var gifts = {'first': 'partridge'};  
assert(gifts['fifth'] == null);
```

Map'teki anahtar-değer çifti sayısını almak için `.length` kullanın:

[VS Code'da aç](#) `cods/1_types/3_collections/15_map_length.dart`

dart

```
var gifts = {'first': 'partridge'};
gifts['fourth'] = 'calling birds';
assert(gifts.length == 2);
```

Derleme zamanı sabiti olan bir map oluşturmak için map literal'inin önüne `const` ekleyin:

[VS Code'da aç](#) `cods/1_types/3_collections/16_const_map.dart`

dart

```
final constantMap = const {2: 'helium', 10: 'neon', 18: 'argon'};

// constantMap[2] = 'Helium'; // Bu satır hataya yol açar.
```

Map'ler hakkında daha fazla bilgi için [dart:core](#) dokümantasyonunun Maps bölümüne bakın.

Koleksiyon elemanları

Bir koleksiyon literal'i bir dizi eleman içerir. Çalışma zamanında her eleman değerlendirilir ve sıfır ya da daha fazla değer üretir; bu değerler ortaya çıkan koleksiyona eklenir. Bu elemanlar iki ana kategoriye ayrılır: yaprak (leaf) elemanlar ve kontrol akışı elemanları.

- Yaprak eleman: Koleksiyon literal'ine tek bir öge ekler.
 - İfade elemanı: Tek bir ifadeyi değerlendirir ve ortaya çıkan değeri koleksiyona ekler.
 - Map girdisi elemanı: Bir anahtar ve değer ifadesi çiftini değerlendirir ve ortaya çıkan girdiyi koleksiyona ekler.
- Kontrol akışı elemanı: Çevreleyen koleksiyona koşullu ya da yinelemeli olarak sıfır ya da daha fazla değer ekler.
 - Null-farkındalıklı (null-aware) eleman: Bir ifadeyi değerlendirir ve sonuç `null` değilse değeri çevreleyen koleksiyona ekler.
 - Spread elemanı: Verilen bir dizi (koleksiyon ifadesi) üzerinde dolaşır ve ortaya çıkan tüm değerleri çevreleyen koleksiyona ekler.
 - Null-farkındalıklı spread elemanı: Spread elemanına benzer, ancak koleksiyonun `null` olmasına izin verir ve null ise hiçbir şey eklemeyiz.
 - If elemanı: Verilen bir koşul ifadesine göre içindeki bir elemanı koşullu olarak değerlendirir; koşul false ise isteğe bağlı olarak başka bir `else` elemanını değerlendirir.
 - For elemanı: Verilen bir iç elemanı yineleyerek tekrar tekrar değerlendirir ve sıfır ya da daha fazla sonuç değeri ekler.

Koleksiyon elemanları hakkında daha fazla bilgi için aşağıdaki bölümlere bakın.

İfade elemanları

Bir ifade elemanı tek bir ifadeyi değerlendirir ve ortaya çıkan değeri koleksiyona ekler. Bu ifade literal'ler, değişkenler, operatörler, fonksiyon çağrıları ve yapıcı çağrıları gibi çeşitli yapıları kapsayabilir.

Bir ifade elemanının koleksiyon içindeki sözdizimi şöyledir:

[VS Code'da aç](#) `cods/1_types/3_collections/17_sozdizimi.dart`

dart

```
<expression>
```

Map girdisi elemanları

Bir map girdisi elemanı bir anahtar ve değer ifadesi çiftini değerlendirir ve ortaya çıkan girdiyi koleksiyona ekler. Bu çiftteki hem anahtar hem de değer ifade olabilir.

Bir map girdisi elemanının koleksiyon içindeki sözdizimi şöyledir:

[VS Code'da aç](#) `cods/1_types/3_collections/18_sozdizimi.dart`

dart

```
<key_expression>: <value_expression>
```

Null-farkındalıklı elemanlar

Null-farkındalıklı bir eleman bir ifadeyi değerlendirir ve sonuç `null` değilse değeri çevreleyen koleksiyona ekler.

↑ Sürüm notu

Null-farkındalıklı koleksiyon elemanları en az 3.8 [dil sürümü](#) gerektirir.

Null-farkındalıklı bir elemanın bir ifade elemanı içindeki sözdizimi şöyledir:

[VS Code'da aç](#) [cods/1_types/3_collections/19_sozdizimi.dart](#)

dart

```
?<expression>
```

Null-farkındalıklı bir elemanın bir map girdisi elemanı içindeki sözdizimi şöyledir:

[VS Code'da aç](#) [cods/1_types/3_collections/20_sozdizimi.dart](#)

dart

```
// anahtar null-farkındalıklı bir elemandır
?<key_expression>: <value_expression>
```

[VS Code'da aç](#) [cods/1_types/3_collections/21_sozdizimi.dart](#)

dart

```
// değer null-farkındalıklı bir elemandır
<key_expression>: ?<value_expression>
```

[VS Code'da aç](#) [cods/1_types/3_collections/22_sozdizimi.dart](#)

dart

```
// anahtar ve değer null-farkındalıklı elemanlardır
?<key_expression>: ?<value_expression>
```

Aşağıdaki örnekte `absentValue` `null` olduğu için `?absentValue` null-farkındalıklı elemanın sonucu `items` adlı listeye eklenmez:

[VS Code'da aç](#) [cods/1_types/3_collections/23_code_sample.dart](#)

dart

```
int? absentValue = null;
int? presentValue = 3;
var items = [
  1,
  ?absentValue,
  ?presentValue,
  absentValue,
  5,
]; // [1, 3, null, 5]
```

Aşağıdaki örnek, map girdisi elemanları içinde null-farkındalıklı elemanları kullanmanın çeşitli yollarını gösterir:

[VS Code'da aç](#) [cods/1_types/3_collections/24_code_sample.dart](#)

dart

```
String? presentKey = 'Apple';
String? absentKey = null;

int? presentValue = 3;
int? absentValue = null;

var itemsA = {presentKey: absentValue}; // {Apple: null}
var itemsB = {presentKey: ?absentValue}; // {}

var itemsC = {absentKey: presentValue}; // {null: 3}
var itemsD = {?absentKey: presentValue}; // {}

var itemsE = {absentKey: absentValue}; // {null: null}
var itemsF = {?absentKey: ?absentValue}; // {}
```

Spread elemanları

Spread elemanı verilen bir dizi üzerinde dolaşır ve ortaya çıkan tüm değerleri çevreleyen koleksiyona ekler.

Bir spread elemanının koleksiyon içindeki sözdizimi şöyledir. Dizi ifadesi, `Iterable`'ı uygulayan (implement) bir nesne veren herhangi bir ifade olabilir:

[VS Code'da aç](#) `cods/1_types/3_collections/25_sozdizimi.dart`

```
...<sequence_expression>
```

dart

Aşağıdaki örnekte `a` adlı listedeki elemanlar `items` adlı listeye eklenir.

[VS Code'da aç](#) `cods/1_types/3_collections/26_code_sample.dart`

```
var a = [1, 2, null, 4];
var items = [0, ...a, 5]; // [0, 1, 2, null, 4, 5]
```

dart

`null` olabilecek bir ifadeyi yayıyorsanız (spread) ve `null`'ı yok saymak (hiç eleman eklememek) istiyorsanız [null-farkındalıklı spread elemanı](#) kullanın.

Spread operatörü hakkında daha fazla bilgi için [Spread operator](#) sayfasına bakın.

Null-farkındalıklı spread elemanları

Null-farkındalıklı spread elemanı spread elemanına benzer, ancak koleksiyonun `null` olmasına izin verir ve null ise hiçbir şey eklemeyiz.

Null-farkındalıklı bir spread elemanının koleksiyon içindeki sözdizimi şöyledir:

[VS Code'da aç](#) `cods/1_types/3_collections/27_sozdizimi.dart`

```
...?<sequence_expression>
```

dart

Aşağıdaki örnekte `a` adlı liste null olduğu için yok sayılır, ancak `b` adlı listedeki elemanlar `items` adlı listeye eklenir. Dikkat edin: Bir koleksiyonun kendisi `null` değilse ama `null` olan elemanlar içeriyorsa, bu `null` elemanlar yine de sonuca eklenir.

[VS Code'da aç](#) `cods/1_types/3_collections/28_sozdizimi.dart`

```
List<int>? a = null;
var b = [1, null, 3];
var items = [0, ...?a, ...?b, 4]; // [0, 1, null, 3, 4]
```

dart

Null güvenliği (null safety) nedeniyle null olabilecek bir değer üzerinde spread işlemi (`...`) yapamazsınız. Aşağıdaki örnek derleme zamanı hatası verir, çünkü `extraOptions` parametresi null olabilir ve `extraOptions` üzerinde kullanılan spread operatörü null-farkındalıklı değildir.

[VS Code'da aç](#) `cods/1_types/3_collections/29_code_sample.dart`

```
statik analiz: başarısız
List<String> buildCommandLine(
  String executable,
  List<String> options, [
  List<String>? extraOptions,
]) {
  return [
    executable,
    ...options,
    ...extraOptions, // <-- Hata
  ];
}

// Kullanım:
// buildCommandLine('dart', ['run', 'my_script.dart'], null);
// Sonuç:
// Derleme zamanı hatası
```

dart

Null olabilen bir koleksiyonu yaymak istiyorsanız null-farkındalıklı spread elemanı kullanın. Aşağıdaki örnek geçerlidir, çünkü `extraOptions` üzerinde null-farkındalıklı spread operatörü kullanılmıştır.

[VS Code'da aç](#) `cods/1_types/3_collections/30_code_sample.dart`

dart

```
List<String> buildCommandLine(
  String executable,
  List<String> options, [
  List<String>? extraOptions,
]) {
  return [
    executable,
    ...options,
    ...?extraOptions, // <-- Artık sorun yok.
  ];
}

// Kullanım:
//   buildCommandLine('dart', ['run', 'my_script.dart'], null);
// Sonuç:
//   [dart, run, my_script.dart]
```

Null-farkındalıklı spread operatörü hakkında daha fazla bilgi için [Spread operator](#) sayfasına bakın.

If elemanları

Bir `if` elemanı, verilen bir koşul ifadesine göre içindeki bir elemanı koşullu olarak değerlendirir; koşul false ise isteğe bağlı olarak başka bir `else` elemanını değerlendirir.

`if` elemanının birkaç sözdizimi çeşidi vardır:

[VS Code'da aç](#) `cods/1_types/3_collections/31_sozdizimi.dart`

dart

```
// bool ifade true ise sonucu ekle.
if (<bool_expression> <result>
```

[VS Code'da aç](#) `cods/1_types/3_collections/32_sozdizimi.dart`

dart

```
// İfade desenle eşleşirse sonucu ekle.
if (<expression> case <pattern>) <result>
```

[VS Code'da aç](#) `cods/1_types/3_collections/33_sozdizimi.dart`

dart

```
// İşlem true sonuç verirse ilk sonucu,
// aksi halde ikinci sonucu ekle.
if (<bool_expression>) <result> else <result>
```

[VS Code'da aç](#) `cods/1_types/3_collections/34_sozdizimi.dart`

dart

```
// İşlem true sonuç verirse ilk sonucu,
// aksi halde ikinci sonucu ekle.
if (<expression> case <pattern>) <result> else <result>
```

Aşağıdaki örnekler, bir koleksiyon içinde boolean bir ifadeyle `if` elemanı kullanmanın çeşitli yollarını gösterir:

[VS Code'da aç](#) `cods/1_types/3_collections/35_code_sample.dart`

dart

```
var includeItem = true;
var items = [0, if (includeItem) 1, 2, 3]; // [0, 1, 2, 3]
```

[VS Code'da aç](#) `cods/1_types/3_collections/36_code_sample.dart`

dart

```
var includeItem = true;
var items = [0, if (!includeItem) 1, 2, 3]; // [0, 2, 3]
```

[VS Code'da aç](#) [cods/1_types/3_collections/37_code_sample.dart](#)

dart

```
var name = 'apple';
var items = [0, if (name == 'orange') 1 else 10, 2, 3]; // [0, 10, 2, 3]
```

[VS Code'da aç](#) [cods/1_types/3_collections/38_code_sample.dart](#)

dart

```
var name = 'apple';
var items = [
  0,
  if (name == 'kiwi') 1 else if (name == 'pear') 10,
  2,
  3,
]; // [0, 2, 3]
```

Aşağıdaki örnekler, bir koleksiyon içinde `case` kısmı olan bir `if` elemanı kullanmanın çeşitli yollarını gösterir:

[VS Code'da aç](#) [cods/1_types/3_collections/39_code_sample.dart](#)

dart

```
Object data = 123;
var typeInfo = [
  if (data case int i) 'Data is an integer: $i',
  if (data case String s) 'Data is a string: $s',
  if (data case bool b) 'Data is a boolean: $b',
  if (data case double d) 'Data is a double: $d',
]; // [Data is an integer: 123]
```

[VS Code'da aç](#) [cods/1_types/3_collections/40_code_sample.dart](#)

dart

```
var word = 'hello';
var items = [
  1,
  if (word case String(length: var wordLength)) wordLength,
  3,
]; // [1, 5, 3]
```

[VS Code'da aç](#) [cods/1_types/3_collections/41_code_sample.dart](#)

dart

```
var orderDetails = ['Apples', 12, ''];
var summary = [
  'Product: ${orderDetails[0]}',
  if (orderDetails case [_, int qty, _]) 'Quantity: $qty',
  if (orderDetails case [_, _, ''])
    'Delivery: Not Started'
  else
    'Delivery: In Progress',
]; // [Product: Apples, Quantity: 12, Delivery: Not Started]
```

Farklı `if` işlemlerini bir `else if` kısmıyla karıştırabilirsiniz. Örneğin:

[VS Code'da aç](#) `cods/1_types/3_collections/42_code_sample.dart`

dart

```
var a = 'apple';
var b = 'orange';
var c = 'mango';
var items = [
  0,
  if (a == 'apple') 1 else if (a case 'mango') 10,
  if (b case 'pear') 2 else if (b == 'mango') 20,
  if (c case 'apple') 3 else if (c case 'mango') 30,
  4,
]; // [0, 1, 30, 4]
```

`if` koşulu hakkında daha fazla bilgi için [if deyimi](#) bölümüne, `if-case` koşulu hakkında daha fazla bilgi için [if-case deyimi](#) bölümüne bakın.

For elemanları

Bir `for` elemanı verilen bir iç elemanı yineleyerek tekrar tekrar değerlendirir ve sıfır ya da daha fazla sonuç değeri ekler.

Bir `for` elemanının koleksiyon içindeki sözdizimi şöyledir:

[VS Code'da aç](#) `cods/1_types/3_collections/43_sozdizimi.dart`

dart

```
for (<expression> in <collection>) <result>
```

[VS Code'da aç](#) `cods/1_types/3_collections/44_sozdizimi.dart`

dart

```
for (<initialization_clause>; <condition_clause>; <increment_clause>) <result>
```

Aşağıdaki örnekler, bir koleksiyon içinde `for` elemanı kullanmanın çeşitli yollarını gösterir:

[VS Code'da aç](#) `cods/1_types/3_collections/45_code_sample.dart`

dart

```
var numbers = [2, 3, 4];
var items = [1, for (var n in numbers) n * n, 7]; // [1, 4, 9, 16, 7]
```

[VS Code'da aç](#) `cods/1_types/3_collections/46_code_sample.dart`

dart

```
var items = [1, for (var x = 5; x > 2; x--) x, 7]; // [1, 5, 4, 3, 7]
```

[VS Code'da aç](#) `cods/1_types/3_collections/47_code_sample.dart`

dart

```
var items = [1, for (var x = 2; x < 4; x++) x, 7]; // [1, 2, 3, 7]
```

`for` döngüsü hakkında daha fazla bilgi için [for döngüleri](#) bölümüne bakın.

Kontrol akışı elemanlarını iç içe kullanma

Kontrol akışı elemanlarını birbirinin içine yerleştirebilirsiniz. Bu, diğer dillerdeki list comprehension'lara güçlü bir alternatiftir.

Aşağıdaki örnekte `numbers` içindeki yalnızca çift sayılar `items`'a dahil edilir.

[VS Code'da aç](#) `cods/1_types/3_collections/48_code_sample.dart`

dart

```
var numbers = [1, 2, 3, 4, 5, 6, 7];
var items = [
  0,
  for (var n in numbers)
    if (n.isEven) n,
  8,
]; // [0, 2, 4, 6, 8]
```

Bir `if` ya da `for` elemanının hemen içinde bir koleksiyon literal'i üzerinde spread kullanmak yaygın ve deyimsel (idiomatic) bir kullanımdır. Örneğin:

[VS Code'da aç](#) `cods/1_types/3_collections/49_code_sample.dart`

dart

```
var items = [
  if (condition) oneThing(),
  if (condition) ...[multiple(), things()],
]; // [oneThing, [multiple_a, multiple_b], things]
```

Her tür elemanı istediğiniz derinlikte iç içe yerleştirebilirsiniz. Aşağıdaki örnekte `if`, `for` ve spread elemanları bir koleksiyon içinde birbirinin içine yerleştirilmiştir:

[VS Code'da aç](#) `cods/1_types/3_collections/50_code_sample.dart`

dart

```
var nestItems = true;
var ys = [1, 2, 3, 4];
var items = [
  if (nestItems) ...[
    for (var x = 0; x < 3; x++)
      for (var y in ys)
        if (x < y) x + y * 10,
  ],
]; // [10, 20, 30, 40, 21, 31, 41, 32, 42]
```

Generic'ler

Dart'ta generic tipler hakkında bilgi edinin.

Temel dizi tipi olan `List`'in API dokümantasyonuna bakarsanız tipin aslında `List<E>` olduğunu görürsünüz. `<...>` gösterimi List'i *generic* (ya da *parametrelî*) bir tip olarak işaretler; yani biçimsel tip parametreleri olan bir tip. [Geleneksel olarak](#) çoğu tip değişkeninin E, T, S, K ve V gibi tek harfli isimleri vardır.

Neden generic kullanılır?

Generic'ler çoğu zaman tip güvenliği için gereklidir, ama faydaları yalnızca kodunuzun çalışmasını sağlamakla sınırlı değildir:

- Generic tipleri doğru belirtmek daha iyi üretilmiş kodla sonuçlanır.
- Generic'leri kod tekrarını azaltmak için kullanabilirsiniz.

Bir listenin yalnızca string içermesini istiyorsanız onu `List<String>` olarak tanımlayabilirsiniz ("string listesi" diye okunur). Böylece siz, diğer programcılar ve araçlarınız, listeye string olmayan bir şey atamanın muhtemelen bir hata olduğunu fark edebilir. İşte bir örnek:

[VS Code'da aç](#) `cods/1_types/4_generics/01_ornek.dart`

```
statik analiz: başarısız
var names = <String>[];
names.addAll(['Seth', 'Kathy', 'Lars']);
names.add(42); // Hata
```

Generic kullanmanın bir başka nedeni kod tekrarını azaltmaktır. Generic'ler, statik analizin avantajlarından yararlanmaya devam ederken tek bir arayüzü ve uygulamayı birçok tip arasında paylaşmanızı sağlar. Örneğin bir nesneyi önbelleğe almak (cache) için bir arayüz oluşturduğunuzu düşünün:

[VS Code'da aç](#) `cods/1_types/4_generics/02_object_cache.dart`

```
abstract class ObjectCache {
  Object getByKey(String key);
  void setByKey(String key, Object value);
}
```

Bu arayüzün string'e özel bir sürümünü istediğinizi fark ediyor ve başka bir arayüz oluşturuyorsunuz:

[VS Code'da aç](#) `cods/1_types/4_generics/03_string_cache.dart`

```
abstract class StringCache {
  String getByKey(String key);
  void setByKey(String key, String value);
}
```

Sonra bu arayüzün sayıya özel bir sürümünü istediğinize karar veriyorsunuz... Gerisini tahmin edebilirsiniz.

Generic tipler sizi tüm bu arayüzleri oluşturma zahmetinden kurtarabilir. Bunun yerine bir tip parametresi alan tek bir arayüz oluşturabilirsiniz:

[VS Code'da aç](#) `cods/1_types/4_generics/04_cache.dart`

```
abstract class Cache<T> {
  T getByKey(String key);
  void setByKey(String key, T value);
}
```

Bu kodda T yer tutucu tiptir. Bunu, bir geliştiricinin ileride tanımlayacağı bir tip olarak düşünebilirsiniz.

Koleksiyon literal'lerini kullanma

List, set ve map literal'leri parametrelili olabilir. Parametrelili literal'ler, daha önce gördüğünüz literal'lerin aynısıdır; tek fark açılış parantezinden önce `<tip>` (list ve set için) ya da `<anahtarTipi, değerTipi>` (map için) eklemenizdir. Tipli literal kullanımına bir örnek:

[VS Code'da aç](#) `cods/1_types/4_generics/05_collection_literals.dart`

dart

```
var names = <String>['Seth', 'Kathy', 'Lars'];
var uniqueNames = <String>{'Seth', 'Kathy', 'Lars'};
var pages = <String, String>{
  'index.html': 'Homepage',
  'robots.txt': 'Hints for web robots',
  'humans.txt': 'We are people, not machines',
};
```

Parametrelili tipleri yapıcılarla kullanma

Bir yapıcı kullanırken bir ya da daha fazla tip belirtmek için tipleri sınıf adının hemen ardından açılı parantez (`<...>`) içine yazın. Örneğin:

[VS Code'da aç](#) `cods/1_types/4_generics/06_constructor_1.dart`

dart

```
var nameSet = Set<String>.of(names);
```

Aşağıdaki kod, anahtarları tam sayı ve değerleri `View` tipinde olan bir `SplayTreeMap` oluşturur:

[VS Code'da aç](#) `cods/1_types/4_generics/07_constructor_2.dart`

dart

```
var views = SplayTreeMap<int, View>();
```

Generic koleksiyonlar ve içerdikleri tipler

Dart'ta generic tipler *somutlaştırılmıştır* (*reified*); yani tip bilgilerini çalışma zamanında da taşırlar. Örneğin bir koleksiyonun tipini test edebilirsiniz:

[VS Code'da aç](#) `cods/1_types/4_generics/08_generic_collections.dart`

dart

```
var names = <String>[];
names.addAll(['Seth', 'Kathy', 'Lars']);
print(names is List<String>); // true
```

Not

Buna karşılık Java'daki generic'ler *silme* (*erasure*) kullanır; yani generic tip parametreleri çalışma zamanında kaldırılır. Java'da bir nesnenin List olup olmadığını test edebilirsiniz, ama `List<String>` olup olmadığını test edemezsiniz.

Parametrelili tipi kısıtlama

Generic bir tip uygularken argüman olarak verilebilecek tipleri sınırlamak, yani argümanın belirli bir tipin alt tipi olmasını zorunlu kılmak isteyebilirsiniz. Bu kısıtlamaya sınır (bound) denir. Bunu `extends` kullanarak yapabilirsiniz.

Yaygın bir kullanım, bir tipi (varsayılan `Object?` yerine) `Object`'in alt tipi yaparak null olamaz hale getirmektir.

[VS Code'da aç](#) `cods/1_types/4_generics/09_non_nullable.dart`

dart

```
class Foo<T extends Object> {
  // Foo'ya T için verilen her tip null olamaz (non-nullable) olmalıdır.
}
```

`extends`'i `Object` dışındaki tiplerle de kullanabilirsiniz. İşte `SomeBaseClass`'ı genişleten bir örnek; böylece `T` tipindeki nesneler üzerinde `SomeBaseClass`'in üyeleri çağrılabilir:

[VS Code'da aç](#) `cods/1_types/4_generics/10_generic.dart`

dart

```
class Foo<T extends SomeBaseClass> {
  // Uygulama buraya gelir...
  String toString() => "Instance of 'Foo<$T>'";
}

class Extender extends SomeBaseClass {
  ...
}
```

Generic argüman olarak `SomeBaseClass`'ı ya da onun herhangi bir alt tipini kullanmak sorun değildir:

[VS Code'da aç](#) `cods/1_types/4_generics/11_somebaseclass_ok.dart`

dart

```
var someBaseClassFoo = Foo<SomeBaseClass>();
var extenderFoo = Foo<Extender>();
```

Hiç generic argüman belirtmemek de sorun değildir:

[VS Code'da aç](#) `cods/1_types/4_generics/12_no_generic_arg_ok.dart`

dart

```
var foo = Foo();
print(foo); // Instance of 'Foo<SomeBaseClass>'
```

`SomeBaseClass` olmayan herhangi bir tip belirtmek hataya yol açar:

[VS Code'da aç](#) `cods/1_types/4_generics/13_ornek.dart`

dart

statik analiz: başarısız

```
var foo = Foo<Object>();
```

Kendine başvuran tip parametresi kısıtlamaları (F-bound'lar)

Parametre tiplerini sınırlarla kısıtlarken sınırın tip parametresinin kendisine başvurmasını sağlayabilirsiniz. Bu, kendine başvuran bir kısıtlama, yani F-bound oluşturur. Örneğin:

[VS Code'da aç](#) `cods/1_types/4_generics/14_f_bound.dart`

dart

```
abstract interface class Comparable<T> {
  int compareTo(T o);
}

int compareAndOffset<T extends Comparable<T>>(T t1, T t2) =>
  t1.compareTo(t2) + 1;

class A implements Comparable<A> {
  @override
  int compareTo(A other) => /*...uygulama...*/ 0;
}

int useIt = compareAndOffset(A(), A());
```

`T extends Comparable<T>` F-bound'u, `T`'nin kendisiyle karşılaştırılabilir olması gerektiği anlamına gelir. Yani `A` yalnızca aynı tipteki diğer örneklerle karşılaştırılabilir.

Generic metotları kullanma

Metotlar ve fonksiyonlar da tip argümanı alabilir:

[VS Code'da aç](#) [cods/1_types/4_generics/15_method.dart](#)

dart

```
T first<T>(List<T> ts) {  
  // Biraz ön hazırlık ya da hata kontrolü yap, sonra...  
  T tmp = ts[0];  
  // Ek kontrol ya da işlem yap...  
  return tmp;  
}
```

Burada `first` üzerindeki generic tip parametresi (`<T>`), `T` tip argümanını birkaç yerde kullanmanıza izin verir:

- Fonksiyonun dönüş tipinde (`T`).
- Bir argümanın tipinde (`List<T>`).
- Bir yerel değişkenin tipinde (`T tmp`).

Typedef'ler

Dart'ta tip takma adları (type alias) hakkında bilgi edinin.

Bir tip takma adı (type alias), `typedef` anahtar kelimesiyle tanımlandığı için genellikle *typedef* olarak anılır ve bir tipe kısa yoldan başvurmanın bir yoludur. `IntList` adlı bir tip takma adını tanımlama ve kullanma örneği:

[VS Code'da aç](#) `cods/1_types/5_typedefs/01_int_list.dart`

dart

```
typedef IntList = List<int>;
IntList il = [1, 2, 3];
```

Bir tip takma adının tip parametreleri olabilir:

[VS Code'da aç](#) `cods/1_types/5_typedefs/02_list_mapper.dart`

dart

```
typedef ListMapper<X> = Map<X, List<X>>;
Map<String, List<String>> m1 = {}; // Uzun hali.
ListMapper<String> m2 = {}; // Aynı şey, ama daha kısa ve net.
```

⬆ Sürüm notu

2.13'ten önce typedef'ler fonksiyon tipleriyle sınırlıydı. Yeni typedef'leri kullanmak en az 2.13 [dil sürümü](#) gerektirir.

Fonksiyonlar için çoğu durumda typedef yerine [satır içi \(inline\) fonksiyon tipleri](#) kullanmanızı öneriyoruz. Yine de fonksiyon typedef'leri işe yarayabilir:

[VS Code'da aç](#) `cods/1_types/5_typedefs/03_compare.dart`

dart

```
typedef Compare<T> = int Function(T a, T b);

int sort(int a, int b) => a - b;

void main() {
  assert(sort is Compare<int>); // True!
}
```

Dart tip sistemi

Sağlam (sound) Dart kodu neden ve nasıl yazılır.

Dart dili tip güvenlidir (type safe): Bir değişkenin değerinin her zaman değişkenin statik tipiyle eşleşmesini sağlamak için statik tip kontrolü ile [çalışma zamanı kontrollerinin](#) bir birleşimini kullanır. Buna bazen sağlam tiplendirme (sound typing) denir. *Tipler* zorunlu olsa da [tip çıkarımı](#) sayesinde tip *belirtileri* isteğe bağlıdır.

Statik tip kontrolünün bir faydası, Dart'ın [statik analizcisini](#) kullanarak hataları derleme zamanında bulabilmektir.

Statik analiz hatalarının çoğunu generic sınıflara tip belirtimi ekleyerek düzeltebilirsiniz. En yaygın generic sınıflar `List<T>` ve `Map<K,V>` koleksiyon tipleridir.

Örneğin aşağıdaki kodda `printInts()` fonksiyonu bir tam sayı listesini yazdırır, `main()` ise bir liste oluşturup onu `printInts()` 'e geçirir.

[VS Code'da aç](#) `cods/1_types/6_type_system/01_opening_example.dart`

statik analiz: başarısız

dart

```
void printInts(List<int> a) => print(a);

void main() {
  final list = [];
  list.add(1);
  list.add('2');
  printInts(list);
}
```

Yukarıdaki kod, `printInts(list)` çağrısında `list` üzerinde (yukarıda vurgulanan) bir tip hatasına yol açar:

error - The argument type 'List<dynamic>' can't be assigned to the parameter type 'List<int>'. - argument_type_not_assignable

Hata, `List<dynamic>` 'ten `List<int>` 'e sağlam olmayan örtük bir dönüşümü (implicit cast) işaret eder. `list` değişkeninin statik tipi `List<dynamic>` 'tir. Bunun nedeni, `var list = []` başlatma tanımının analizciye `dynamic` 'ten daha belirli bir tip argümanı çıkarması için yeterli bilgi vermemesidir. `printInts()` fonksiyonu `List<int>` tipinde bir parametre beklediği için tipler uyuşmaz.

Liste oluşturulurken (aşağıda vurgulanan) bir tip belirtimi (`<int>`) eklendiğinde, analizci bir string argümanının `int` parametresine atanamayacağından şikâyet eder. `list.add('2')` içindeki tırnakları kaldırmak, statik analizden geçen ve hiçbir hata ya da uyarı vermeden çalışan bir kod ortaya çıkarır.

[VS Code'da aç](#) `cods/1_types/6_type_system/02_opening_example.dart`

statik analiz: başarılı

dart

```
void printInts(List<int> a) => print(a);

void main() {
  final list = <int>[];
  list.add(1);
  list.add(2);
  printInts(list);
}
```

[DartPad'de deneyin.](#)

Sağlamlık (soundness) nedir?

Sağlamlık, programınızın belirli geçersiz durumlara düşmemesini sağlamakla ilgilidir. Sağlam bir *tip sistemi*, bir ifadenin statik tipiyle eşleşmeyen bir değer üretmesi durumuna asla düşmeyeceğiniz anlamına gelir. Örneğin bir ifadenin statik tipi `String` ise, çalışma zamanında onu değerlendirdiğinizde yalnızca bir string alacağınız garantidir.

Dart'ın tip sistemi, Java ve C#'taki tip sistemleri gibi sağlamdır. Bu sağlamlığı statik kontrol (derleme zamanı hataları) ile çalışma zamanı kontrollerinin birleşimiyle uygular. Örneğin bir `String` 'i `int` 'e atamak derleme zamanı hatasıdır. Bir nesneyi `as String` ile `String` 'e dönüştürmek (cast), nesne bir `String` değilse çalışma zamanı hatasıyla başarısız olur.

Sağlamlığın faydaları

Sağlam bir tip sisteminin birçok faydası vardır:

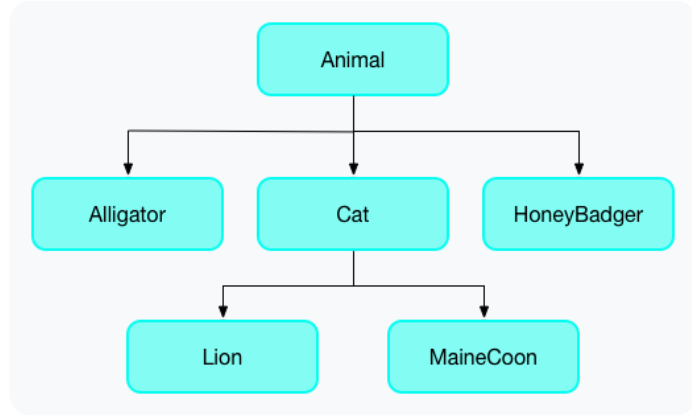
- Tiple ilgili hataları derleme zamanında ortaya çıkarma.
Sağlam bir tip sistemi, kodu tipleri konusunda net olmaya zorlar; böylece çalışma zamanında bulunması zor olabilecek tipte ilgili hatalar derleme zamanında ortaya çıkar.
- Daha okunabilir kod.
Bir değerin gerçekten belirtilen tipte olduğuna güvenebildiğiniz için kod daha kolay okunur. Sağlam Dart'ta tipler yalan söyleyemez.
- Bakımı daha kolay kod.
Sağlam bir tip sistemiyle, bir kod parçasını değiştirdiğinizde tip sistemi sizi bu yüzden bozulan diğer kod parçaları hakkında uyarabilir.
- Daha iyi önceden derleme (AOT, ahead of time).
AOT derleme tipler olmadan da mümkündür, ancak üretilen kod çok daha az verimli olur.

Statik analizden geçmek için ipuçları

Statik tiplerle ilgili kuralların çoğu kolay anlaşılır. İşte daha az belirgin olan kurallardan bazıları:

- Metotları override ederken sağlam dönüş tipleri kullanın.
- Metotları override ederken sağlam parametre tipleri kullanın.
- Dinamik bir listeyi tipli bir liste olarak kullanmayın.

Bu kuralları aşağıdaki tip hiyerarşisini kullanan örneklerle ayrıntılı olarak inceleyelim:



Metotları override ederken sağlam dönüş tipleri kullanın

Bir alt sınıftaki metodun dönüş tipi, üst sınıftaki metodun dönüş tipiyle aynı tip ya da onun bir alt tipi olmalıdır. `Animal` sınıfındaki getter metodunu düşünün:

[VS Code'da aç](#) `cods/1_types/6_type_system/03_animal.dart`

dart

```
class Animal {  
  void chase(Animal a) {  
    ...  
  }  
  Animal get parent => ...  
}
```

`parent` getter metodu bir `Animal` döndürür. `HoneyBadger` alt sınıfında `getter`'ın dönüş tipini `HoneyBadger` (ya da `Animal`'ın herhangi bir başka alt tipi) ile değiştirebilirsiniz, ama ilgisiz bir tipe izin verilmez.

[VS Code'da aç](#) `cods/1_types/6_type_system/04_honeybadger.dart`

statik analiz: başarılı

dart

```
class HoneyBadger extends Animal {  
  @override  
  void chase(Animal a) {  
    ...  
  }  
  
  @override  
  HoneyBadger get parent => ...  
}
```

[VS Code'da aç](#) `cods/1_types/6_type_system/05_honeybadger.dart`

statik analiz: başarısız

dart

```
class HoneyBadger extends Animal {  
  @override  
  void chase(Animal a) {  
    ...  
  }  
  
  @override  
  Root get parent => ...  
}
```

Metotları override ederken sağlam parametre tipleri kullanın

Override edilen bir metodun parametresi, üst sınıftaki karşılık gelen parametreyle aynı tipte ya da onun bir üst tipinde olmalıdır. Tipi orijinal parametrenin bir alt tipiyle değiştirerek parametre tipini "daraltmayın".

Not

Alt tip kullanmak için geçerli bir nedeniniz varsa `covariant` anahtar kelimesini kullanabilirsiniz.

`Animal` sınıfının `chase(Animal)` metodunu düşünün:

[VS Code'da aç](#) `cods/1_types/6_type_system/06_animal.dart`

dart

```
class Animal {  
  void chase(Animal a) {  
    ...  
  }  
  Animal get parent => ...  
}
```

`chase()` metodu bir `Animal` alır. Bir `HoneyBadger` her şeyi kovalar. `chase()` metodunu her şeyi (`Object`) alacak şekilde override etmek sorun değildir.

[VS Code'da aç](#) `cods/1_types/6_type_system/07_chase_object.dart`

dart

statik analiz: başarılı

```
class HoneyBadger extends Animal {  
  @override  
  void chase(Object a) {  
    ...  
  }  
  
  @override  
  Animal get parent => ...  
}
```

Aşağıdaki kod, `chase()` metodundaki parametreyi `Animal` 'dan `Animal` 'ın bir alt sınıfı olan `Mouse` 'a daraltır.

[VS Code'da aç](#) `cods/1_types/6_type_system/08_chase_mouse.dart`

```
statik analiz: başarısız dart
class Mouse extends Animal {
  ...
}

class Cat extends Animal {
  @override
  void chase(Mouse a) {
    ...
  }
}
```

Bu kod tip güvenli değildir, çünkü o zaman bir kedi tanımlayıp onu bir timsahın peşine göndermek mümkün olurdu:

[VS Code'da aç](#) `cods/1_types/6_type_system/09_would_not_be_type_safe.dart`

```
Animal a = Cat();
a.chase(Alligator()); // Ne tip açısından ne de kedi açısından güvenli.
```

Dinamik bir listeyi tipli bir liste olarak kullanmayın

İçinde farklı türde şeyler bulunan bir liste istediğinizde `dynamic` bir liste iyi bir seçimdir. Ancak `dynamic` bir listeyi tipli bir liste olarak kullanamazsınız.

Bu kural generic tiplerin örnekleri için de geçerlidir.

Aşağıdaki kod `Dog` 'lardan oluşan `dynamic` bir liste oluşturur ve onu `Cat` tipinde bir listeye atar; bu da statik analiz sırasında hata üretir.

[VS Code'da aç](#) `cods/1_types/6_type_system/10_invalid_dynamic_list.dart`

```
statik analiz: başarısız dart
void main() {
  List<Cat> foo = <dynamic>[Dog()]; // Hata
  List<dynamic> bar = <dynamic>[Dog(), Cat()]; // Sorun yok
}
```

Çalışma zamanı kontrolleri

Çalışma zamanı kontrolleri, derleme zamanında tespit edilemeyen tip güvenliği sorunlarıyla ilgilenir.

Örneğin aşağıdaki kod çalışma zamanında bir istisna fırlatır, çünkü köpek listesini kedi listesine dönüştürmek bir hatadır:

[VS Code'da aç](#) `cods/1_types/6_type_system/11_runtime_checks.dart`

```
çalışma zamanı: başarısız dart
void main() {
  List<Animal> animals = <Dog>[Dog()];
  List<Cat> cats = animals as List<Cat>;
}
```

`dynamic` 'ten örtük aşağı dönüşümler (downcast)

Statik tipi `dynamic` olan ifadeler örtük olarak daha belirli bir tipe dönüştürülebilir. Gerçek tip eşleşmezse dönüşüm çalışma zamanında hata fırlatır.

Aşağıdaki `assumeString` metodunu düşünün:

[VS Code'da aç](#) `cods/1_types/6_type_system/12_downcast_check.dart`

```
statik analiz: başarılı dart
int assumeString(dynamic object) {
  String string = object; // `object`'in bir `String` olduğunu çalışma zamanında kontrol et.
  return string.length;
}
```

Bu örnekte `object` bir `String` ise dönüşüm başarılı olur. `int` gibi `String`'in alt tipi olmayan bir şeyse bir `TypeError` fırlatılır:

[VS Code'da aç](#) `cods/1_types/6_type_system/13_fail_downcast_check.dart`

çalışma zamanı: başarısız

dart

```
final length = assumeString(1);
```

İpucu

`dynamic`'ten örtük aşağı dönüşümleri önlemek ve bu sorundan kaçınmak için analizcinin *strict casts* (katı dönüşümler) modunu açmayı düşünün.

analysis_options.yaml

yaml

```
analyzer:
  language:
    strict-casts: true
```

Analizcinin davranışını özelleştirme hakkında daha fazla bilgi için [Customizing static analysis](#) sayfasına göz atın.

Tip çıkarımı

Analizci; alanların, metotların, yerel değişkenlerin ve çoğu generic tip argümanının tiplerini çıkarabilir. Analizcinin belirli bir tipi çıkarmak için yeterli bilgisi olmadığında `dynamic` tipini kullanır.

Tip çıkarımının generic'lerle nasıl çalıştığına dair bir örnek: Bu örnekte `arguments` adlı bir değişken, string anahtarları çeşitli tipteki değerlerle eşleştiren bir map tutar.

Değişkenin tipini açıkça yazarsanız şöyle yazabilirsiniz:

[VS Code'da aç](#) `cods/1_types/6_type_system/14_type_inference_1_orig.dart`

dart

```
Map<String, Object?> arguments = {'argA': 'hello', 'argB': 42};
```

Alternatif olarak `var` ya da `final` kullanıp tipi Dart'ın çıkarmasına izin verebilirsiniz:

[VS Code'da aç](#) `cods/1_types/6_type_system/15_type_inference_1.dart`

dart

```
var arguments = {'argA': 'hello', 'argB': 42}; // Map<String, Object>
```

Map literal'i tipini girdilerinden çıkarır, değişken de tipini map literal'inin tipinden çıkarır. Bu map'te anahtarların ikisi de string'dir, ama değerler farklı tiptedir (üst sınırı `Object` olan `String` ve `int`). Bu yüzden map literal'inin tipi `Map<String, Object>` olur, `arguments` değişkeninin tipi de öyle.

Alan ve metot çıkarımı

Tipi belirtilmemiş ve üst sınıftan bir alanı ya da metodu override eden bir alan ya da metot, üst sınıftaki metodun ya da alanın tipini miras alır.

Tanımlanmış ya da miras alınmış bir tipi olmayan ama bir başlangıç değeriyle tanımlanan bir alanın tipi, başlangıç değerine göre çıkarılır.

Statik alan çıkarımı

Statik alanların ve değişkenlerin tipleri başlatıcılarından (initializer) çıkarılır. Çıkarımın bir döngüyle karşılaşırsa başarısız olduğunu unutmayın (yani değişkenin tipini çıkarmak, o değişkenin tipini bilmeye bağlıysa).

Yerel değişken çıkarımı

Yerel değişkenlerin tipleri, varsa başlatıcılarından çıkarılır. Sonraki atamalar hesaba katılmaz. Bu, gereğinden fazla kesin bir tipin çıkarılmasına yol açabilir. Böyle bir durumda tip belirtimi ekleyebilirsiniz.

[VS Code'da aç](#) `cods/1_types/6_type_system/16_local_var_type_inference_error.dart`

dart

statik analiz: başarısız

```
var x = 3; // x'in tipi int olarak çıkarılır.
x = 4.0;
```

[VS Code'da aç](#) `cods/1_types/6_type_system/17_local_var_type_inference_ok.dart`

```
statik analiz: başarılı

num y = 3; // Bir num, double ya da int olabilir.
y = 4.0;
```

dart

Tip argümanı çıkarımı

Yapıcı çağrılarına ve [generic metot](#) çağrılarına verilen tip argümanları, kullanıldıkları bağlamdan gelen aşağı yönlü bilgi ile yapıcıya ya da generic metoda verilen argümanlardan gelen yukarı yönlü bilginin birleşimine göre çıkarılır. Çıkarım istediğiniz ya da beklediğiniz şeyi yapmıyorsa tip argümanlarını her zaman açıkça belirtebilirsiniz.

[VS Code'da aç](#) cods/1_types/6_type_system/18_type_arg_inference.dart

```
statik analiz: başarılı

// <int>[] yazmışsınız gibi çıkarılır.
List<int> listOfInt = [];

// <double>[3.0] yazmışsınız gibi çıkarılır.
var listOfDouble = [3.0];

// Iterable<int> olarak çıkarılır.
var ints = listOfDouble.map((x) => x.toInt());
```

dart

Son örnekte `x`'in tipi aşağı yönlü bilgi kullanılarak `double` olarak çıkarılır. Closure'ın dönüş tipi ise yukarı yönlü bilgi kullanılarak `int` olarak çıkarılır. Dart bu dönüş tipini `map()` metodunun tip argümanını (`<int>`) çıkarırken yukarı yönlü bilgi olarak kullanır.

Sınırları kullanarak çıkarım

⬆ Sürüm notu

Sınırları kullanarak çıkarım en az 3.7.0 [dil sürümü](#) gerektirir.

Sınırları kullanarak çıkarım özelliğiyle Dart'ın tip çıkarım algoritması, yalnızca en iyi tahmine dayalı yaklaşımlarla değil, mevcut kısıtlamaları tanımlanmış tip sınırlarıyla birleştirilerek kısıtlamalar üretir.

Bu özellikle [F-sınırlı](#) tipler için önemlidir; sınırları kullanarak çıkarım, aşağıdaki örnekte `X`'in `B`'ye bağlanabileceğini doğru şekilde çıkarır. Bu özellik olmadan tip argümanı açıkça belirtilmelidir: `f<B>(C())`:

[VS Code'da aç](#) cods/1_types/6_type_system/19_inference_using_bounds.dart

```
class A<X extends A<X>> {}

class B extends A<B> {}

class C extends B {}

void f<X extends A<X>>(X x) {}

void main() {
  f(B()); // Sorun yok.

  // Sorun yok. Sınırlar kullanılmadan, en iyi tahmine dayanan çıkarım
  // 'C''nin 'A<C>'nin alt tipi olmadığını gördükten sonra başarısız olurdu.
  f(C());

  f<B>(C()); // Sorun yok.
}
```

dart

Dart'taki `int` ya da `num` gibi gündelik tiplerle daha gerçekçi bir örnek:

[VS Code'da aç](#) `cods/1_types/6_type_system/20_inference_using_bounds_2.dart`

dart

```
X max<X extends Comparable<X>>(X x1, X x2) => x1.compareTo(x2) > 0 ? x1 : x2;

void main() {
  // Bu özellikle 'max<num>(3, 7)' olarak çıkarılır, özellik olmadan başarısız olur.
  max(3, 7);
}
```

Sınırları kullanarak çıkarımla Dart, tip argümanlarını *ayrıştırılabilir* ve generic bir tip parametresinin sınırından tip bilgisi çıkarabilir. Bu, aşağıdaki örnekteki `f` gibi fonksiyonların hem belirli iterable tipini (`List` ya da `Set`) *hem de* eleman tipini korumasını sağlar. Sınırları kullanarak çıkarımdan önce bu, tip güvenliğini ya da belirli tip bilgisini kaybetmeden mümkün değildi.

[VS Code'da aç](#) `cods/1_types/6_type_system/21_ornek.dart`

dart

```
(X, Y) f<X extends Iterable<Y>, Y>(X x) => (x, x.first);

void main() {
  var (myList, myInt) = f([1]);
  myInt.whatever; // Derleme zamanı hatası, 'myInt'in tipi 'int'.

  var (mySet, myString) = f({'Hello!'});
  mySet.union({}); // Çalışır, 'mySet'in tipi 'Set<String>'.
}
```

Sınırları kullanarak çıkarım olmasaydı `myInt`'in tipi `dynamic` olurdu. Önceki çıkarım algoritması hatalı `myInt.whatever` ifadesini derleme zamanında yakalayamaz, bunun yerine çalışma zamanında hata fırlatırdı. Tersine, sınırları kullanarak çıkarım olmadan `mySet.union({})` derleme zamanı hatası olurdu, çünkü önceki algoritma `mySet`'in bir `Set` olduğu bilgisini koruyamazdı.

Sınırları kullanarak çıkarım algoritması hakkında daha fazla bilgi için [tasarım belgesini](#) okuyun.

Tiplerin yerine koyulması

Bir metodu override ettiğinizde, bir tipteki bir şeyi (eski metotta) yeni bir tipe sahip olabilecek bir şeyle (yeni metotta) değiştirirsiniz. Benzer şekilde, bir fonksiyona argüman geçirdiğinizde, bir tipe sahip bir şeyi (tanımlı tipi olan bir parametreyi) başka bir tipe sahip bir şeyle (gerçek argümanla) değiştirirsiniz. Bir tipe sahip bir şeyi ne zaman bir alt tipe ya da üst tipe sahip bir şeyle değiştirebilirsiniz?

Tipleri yerine koyarken *tüketiciler* (*consumer*) ve *üreticiler* (*producer*) açısından düşünmek işe yarar. Tüketici bir tipi alır, üretici bir tip üretir.

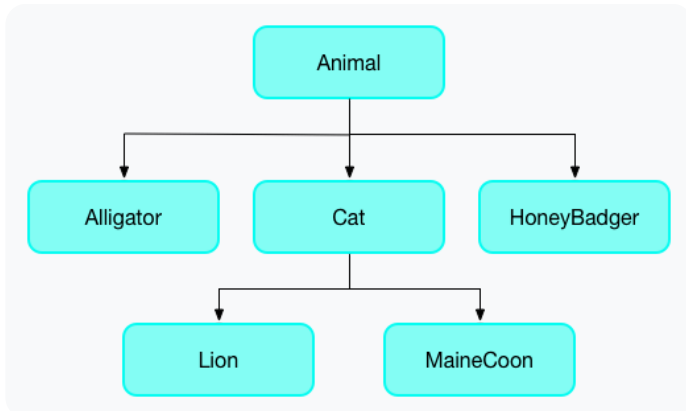
Bir tüketicinin tipini bir üst tipe, bir üreticinin tipini ise bir alt tipe değiştirebilirsiniz.

Basit tip atamasına ve generic tiplerle atamaya ilişkin örneklerle bakalım.

Basit tip ataması

Nesneleri nesnelere atarken bir tipi ne zaman farklı bir tipe değiştirebilirsiniz? Cevap, nesnenin bir tüketici mi yoksa bir üretici mi olduğuna bağlıdır.

Aşağıdaki tip hiyerarşisini düşünün:



`Cat c` 'nin bir *tüketici*, `Cat()` 'in ise bir *üretici* olduğu şu basit atamayı düşünün:

[VS Code'da aç](#) `cods/1_types/6_type_system/22_cat_cat_ok.dart`

```
Cat c = Cat();
```

dart

Tüketen konumda, belirli bir tipi (`Cat`) tüketen bir şeyi her şeyi (`Animal`) tüketen bir şeyle değiştirmek güvenlidir; bu yüzden `Cat c` 'yi `Animal c` ile değiştirmeye izin verilir, çünkü `Animal`, `Cat` 'in bir üst tipidir.

[VS Code'da aç](#) `cods/1_types/6_type_system/23_animal_cat_ok.dart`

```
statik analiz: başarılı  
Animal c = Cat();
```

dart

Ancak `Cat c` 'yi `MaineCoon c` ile değiştirmek tip güvenliğini bozar, çünkü üst sınıf `Lion` gibi farklı davranışları olan bir `Cat` tipi sağlayabilir:

[VS Code'da aç](#) `cods/1_types/6_type_system/24_mainecoon_cat_err.dart`

```
statik analiz: başarısız  
MaineCoon c = Cat();
```

dart

Üreten konumda, bir tip (`Cat`) üreten bir şeyi daha belirli bir tipe (`MaineCoon`) değiştirmek güvenlidir. Bu yüzden aşağıdakine izin verilir:

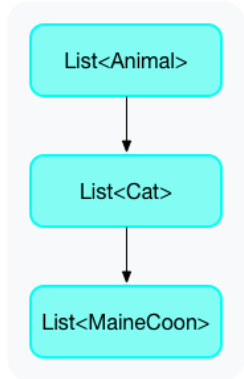
[VS Code'da aç](#) `cods/1_types/6_type_system/25_cat_mainecoon_ok.dart`

```
statik analiz: başarılı  
Cat c = MaineCoon();
```

dart

Generic tip ataması

Generic tipler için kurallar aynı mı? Evet. Hayvan listelerinin hiyerarşisini düşünün: `Cat` 'lerden oluşan bir `List`, `Animal` 'lardan oluşan bir `List` 'in alt tipi, `MaineCoon` 'lardan oluşan bir `List` 'in ise üst tipidir:



Aşağıdaki örnekte `myCats` 'e bir `MaineCoon` listesi atayabilirsiniz, çünkü `List<MaineCoon>`, `List<Cat>` 'in bir alt tipidir:

[VS Code'da aç](#) `cods/1_types/6_type_system/26_generic_type_assignment_mainecoon.dart`

```
statik analiz: başarılı  
List<MaineCoon> myMaineCoons = ...  
List<Cat> myCats = myMaineCoons;
```

dart

Peki ters yönde ne olur? Bir `Animal` listesini bir `List<Cat>` 'e atayabilir misiniz?

[VS Code'da aç](#) `cods/1_types/6_type_system/27_generic_type_assignment_animal.dart`

```
statik analiz: başarısız  
List<Animal> myAnimals = ...  
List<Cat> myCats = myAnimals;
```

dart

Bu atama statik analizden geçmez, çünkü `Animal` gibi `dynamic` olmayan tiplerden izin verilmeyen örtük bir aşağı dönüşüm oluşturur.

Bu tür bir kodun statik analizden geçmesi için açık bir dönüşüm (explicit cast) kullanabilirsiniz.

VS Code'da aç [cods/1_types/6_type_system/28_generic_type_assignment_implied_cast.dart](#)

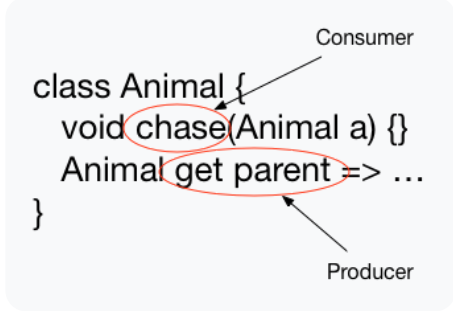
dart

```
List<Animal> myAnimals = ...  
List<Cat> myCats = myAnimals as List<Cat>;
```

Yine de açık bir dönüşüm, dönüştürülen listenin (`myAnimals`) gerçek tipine bağlı olarak çalışma zamanında başarısız olabilir.

Metotlar

Bir metodu override ederken üretici ve tüketici kuralları yine geçerlidir. Örneğin:



Bir tüketici için (`chase(Animal)` metodu gibi) parametre tipini bir üst tipe değiştirebilirsiniz. Bir üretici için (`parent` getter metodu gibi) dönüş tipini bir alt tipe değiştirebilirsiniz.

Daha fazla bilgi için [Metotları override ederken sağlam dönüş tipleri kullanın](#) ve [Metotları override ederken sağlam parametre tipleri kullanın](#) bölümlerine bakın.

Kovaryant parametreler

Bazı (nadiren kullanılan) kodlama kalıpları, bir parametrenin tipini bir alt tipe override ederek tipi daraltmaya dayanır; bu geçersizdir. Bu durumda analizciye bunu bilerek yaptığınızı söylemek için `covariant` anahtar kelimesini kullanabilirsiniz. Bu, statik hatayı kaldırır ve bunun yerine geçersiz argüman tipini çalışma zamanında kontrol eder.

Aşağıda `covariant`'ı nasıl kullanabileceğiniz gösteriliyor:

VS Code'da aç [cods/1_types/6_type_system/29_ornek.dart](#)

dart

```
statik analiz: başarılı  
  
class Animal {  
  void chase(Animal x) {  
    ...  
  }  
}  
  
class Mouse extends Animal {  
  ...  
}  
  
class Cat extends Animal {  
  @override  
  void chase(covariant Mouse x) {  
    ...  
  }  
}
```

Bu örnek `covariant`'ın alt tipe kullanımını gösterse de `covariant` anahtar kelimesi üst sınıftaki ya da alt sınıftaki metoda yerleştirilebilir. Genellikle en iyi yer üst sınıftaki metottur. `covariant` anahtar kelimesi tek bir parametreye uygulanır ve setter'larda ve alanlarda da desteklenir.

Diğer kaynaklar

Aşağıdaki kaynaklarda sağlam Dart hakkında daha fazla bilgi bulunur:

- [Fixing type promotion failures](#) - Tip yükseltme (type promotion) hatalarını anlayın ve nasıl düzelteceğinizi öğrenin.

- [Sound null safety](#) - Sağlam null güvenliğiyle kod yazmayı öğrenin.
- [Customizing static analysis](#) - Bir analiz seçenekleri dosyası kullanarak analizciyi ve linter'ı kurma ve özelleştirme.

Desenler

Dart'taki desenlerin (pattern) özeti.

↑ Sürüm notu

Desenler en az 3.0 [dil sürümü](#) gerektirir.

Desenler (pattern), deyimler (statement) ve ifadeler (expression) gibi Dart dilinde bir sözdizimi kategorisidir. Bir desen, gerçek değerlerle eşleştirilebilecek bir değer kümesinin şeklini temsil eder.

Bu sayfada şunlar anlatılıyor:

- Desenlerin ne yaptığı.
- Dart kodunda desenlerin nerelerde kullanılabildiği.
- Desenlerin yaygın kullanım alanları.

Farklı desen türleri hakkında bilgi için [desen tipleri](#) sayfasını ziyaret edin.

Desenler ne yapar?

Genel olarak bir desen, bağlama ve desenin şekline göre bir değeri **eşleştirebilir**, bir değeri **ayrıştırabilir** ya da her ikisini birden yapabilir.

Önce *desen eşleştirme*, verilen bir değerın şunları sağlayıp sağlamadığını kontrol etmenize olanak tanır:

- Belirli bir şekle sahip olma.
- Belirli bir sabit olma.
- Başka bir şeye eşit olma.
- Belirli bir tipte olma.

Ardından *desen ayrıştırma* (*destructuring*), o değeri parçalarına ayırmak için kullanışlı, bildirimsel (declarative) bir sözdizimi sunar. Aynı desen, bu süreçte değişkenleri o parçaların bir kısmına ya da tamamına bağlamanıza da olanak tanıyabilir.

Eşleştirme

Bir desen, değerın beklediğiniz biçimde olup olmadığını belirlemek için her zaman bir değere karşı test edilir. Başka bir deyişle, değerin desenle *eşleşip eşleşmediğini* kontrol edersiniz.

Neyin eşleşme sayılacağı [hangi tür deseni](#) kullandığınıza bağlıdır. Örneğin bir sabit deseni, değer desenin sabitine eşitse eşleşir:

[VS Code'da aç](#) `cods/2_patterns/1_patterns/01_constant_pattern.dart`

dart

```
switch (number) {
  // Sabit desen 1 == number ise eşleşir.
  case 1:
    print('one');
}
```

Birçok desen alt desenlerden yararlanır; bunlara bazen sırasıyla *dış* ve *iç* desenler denir. Desenler alt desenleri üzerinde özyinelemeli (recursive) olarak eşleşir. Örneğin herhangi bir [koleksiyon tipi](#) deseninin tek tek alanları [değişken desenleri](#) ya da [sabit desenleri](#) olabilir:

[VS Code'da aç](#) `cods/2_patterns/1_patterns/02_list_pattern.dart`

dart

```
const a = 'a';
const b = 'b';
switch (obj) {
  // [a, b] list deseni, obj iki alanlı bir liste ise önce obj ile eşleşir,
  // sonra alanları 'a' ve 'b' sabit alt desenleriyle eşleşirse.
  case [a, b]:
    print('$a, $b');
}
```

Eşleşen bir değerin bazı kısımlarını yok saymak için yer tutucu olarak [joker \(wildcard\) desen](#) kullanabilirsiniz. List desenlerinde ise [rest elemanı](#) kullanabilirsiniz.

Ayrıştırma

Bir nesne ile desen eşleştiğinde desen nesnenin verilerine erişebilir ve onları parçalar halinde çıkarabilir. Başka bir deyişle desen nesneyi *ayrıştırır*.

[VS Code'da aç](#) [cods/2_patterns/1_patterns/03_list_pattern.dart](#)

dart

```
var numList = [1, 2, 3];
// [a, b, c] list deseni numList'teki üç elemanı ayrıştırır...
var [a, b, c] = numList;
// ...ve onları yeni değişkenlere atar.
print(a + b + c);
```

Bir ayrıştırma deseninin içine [her tür deseni](#) yerleştirebilirsiniz. Örneğin bu case deseni, ilk elemanı `'a'` ya da `'b'` olan iki elemanlı bir listeyi eşleştirir ve ayrıştırır:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/04_nested_pattern.dart](#)

dart

```
switch (list) {
  case ['a' || 'b', var c]:
    print(c);
}
```

Desenlerin kullanılabildiği yerler

Desenleri Dart dilinde birkaç yerde kullanabilirsiniz:

- Yerel değişken [tanımları](#) ve [atamaları](#)
- [for](#) ve [for-in döngüleri](#)
- [if-case](#) ve [switch-case](#)
- [Koleksiyon literal'lerinde](#) kontrol akışı

Bu bölümde desenlerle eşleştirme ve ayrıştırmanın yaygın kullanım alanları anlatılıyor.

Değişken tanımı

Dart'ın yerel değişken tanımına izin verdiği her yerde *desen değişken tanımı* kullanabilirsiniz. Desen, tanımın sağındaki değerle eşleştirilir. Eşleştikten sonra değeri ayrıştırır ve yeni yerel değişkenlere bağlar:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/05_variable_declaration.dart](#)

dart

```
// Yeni a, b ve c değişkenlerini tanımlar.
var (a, [b, c]) = ('str', [1, 2]);
```

Bir desen değişken tanımı `var` ya da `final` ile başlamalı, ardından bir desen gelmelidir.

Değişken ataması

Bir *değişken atama deseni* bir atamanın sol tarafında yer alır. Önce eşleşen nesneyi ayrıştırır. Sonra değerleri yenilerini bağlamak yerine *var olan* değişkenlere atar.

Üçüncü bir geçici değişken tanımlamadan iki değişkenin değerlerini yer değiştirmek için bir değişken atama deseni kullanın:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/06_variable_assignment.dart](#)

dart

```
var (a, b) = ('left', 'right');
(b, a) = (a, b); // Yer değiştir.
print('$a $b'); // "right left" yazdırır.
```

Switch deyimleri ve ifadeleri

Her case cümlesi bir desen içerir. Bu, [switch deyimleri](#) ve [ifadeleri](#) ile [if-case deyimleri](#) için geçerlidir. Bir case içinde [her tür deseni](#) kullanabilirsiniz.

Case desenleri [çürütülebilirdir \(refutable\)](#), yani bir değere karşı test edilebilirler. Kontrol akışının şunlardan birini yapmasına izin verirler:

- Üzerinde switch yapılan nesneyi eşleştirip ayırtırmak.
- Nesne eşleşmezse çalışmaya devam etmek.

Bir desenin bir case içinde ayırtırdığı değerler yerel değişken olur. Kapsamları yalnızca o case'in gövdesidir.

[VS Code'da aç](#) cods/2_patterns/1_patterns/07_switch_statement.dart

dart

```
switch (obj) {  
  // 1 == obj ise eşleşir.  
  case 1:  
    print('one');  
  
  // obj'nin değeri 'first' ve 'last' sabit  
  // değerleri arasındaysa eşleşir.  
  case >= first && <= last:  
    print('in range');  
  
  // obj iki alanlı bir record ise eşleşir,  
  // sonra alanları 'a' ve 'b'ye atar.  
  case (var a, var b):  
    print('a = $a, b = $b');  
  
  default:  
}
```

Mantıksal-veya desenleri, switch ifadelerinde ya da deyimlerinde birden fazla case'in aynı gövdeyi paylaşması için kullanışlıdır:

[VS Code'da aç](#) cods/2_patterns/1_patterns/08_or_share_body.dart

dart

```
var isPrimary = switch (color) {  
  Color.red || Color.yellow || Color.blue => true,  
  _ => false,  
};
```

Switch deyimlerinde birden fazla case, **mantıksal-veya deseni kullanmadan** da aynı gövdeyi paylaşabilir; ancak bu desenler birden fazla case'in aynı **koruma ifadesini (guard)** paylaşmasını sağlamak için hâlâ benzersiz şekilde kullanışlıdır:

[VS Code'da aç](#) cods/2_patterns/1_patterns/09_or_share_guard.dart

dart

```
switch (shape) {  
  case Square(size: var s) || Circle(size: var s) when s > 0:  
    print('Non-empty symmetric shape');  
}
```

Koruma ifadeleri, case'in bir parçası olarak keyfi bir koşulu değerlendirir; koşul false olsa bile switch'ten çıkmaz (case gövdesinde bir `if` deyimini kullanmak buna yol açardı).

[VS Code'da aç](#) cods/2_patterns/1_patterns/10_guard.dart

dart

```
switch (pair) {  
  case (int a, int b):  
    if (a > b) print('First element greater');  
  // false ise hiçbir şey yazdırmaz ve switch'ten çıkar.  
  case (int a, int b) when a > b:  
    // false ise hiçbir şey yazdırmaz ama sonraki case'e geçer.  
    print('First element greater');  
  case (int a, int b):  
    print('First element not greater');  
}
```

For ve for-in döngüleri

Bir koleksiyondaki değerler üzerinde dolaşmak ve onları ayrıştırmak için [for ve for-in döngülerinde](#) desen kullanabilirsiniz.

Bu örnek, bir `<Map>.entries` çağrısının döndürdüğü `MapEntry` nesnelerini ayrıştırmak için bir for-in döngüsünde [nesne ayrıştırma](#) kullanır:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/11_for_in_pattern.dart](#)

```
Map<String, int> hist = {'a': 23, 'b': 100};

for (var MapEntry(key: key, value: count) in hist.entries) {
  print('$key occurred $count times');
}
```

Nesne deseni, `hist.entries`'in `MapEntry` isimli tipinde olduğunu kontrol eder, ardından `key` ve `value` isimli alan alt desenlerine iner. Her yinelemede `MapEntry` üzerinde `key` getter'ını ve `value` getter'ını çağırır ve sonuçları sırasıyla `key` ve `count` yerel değişkenlerine bağlar.

Bir getter çağrısının sonucunu aynı isimli bir değişkene bağlamak yaygın bir kullanımdır; bu yüzden nesne desenleri getter adını [değişken alt deseninden](#) de çıkarabilir. Bu, değişken desenini `key: key` gibi gereksiz tekrarlı bir halden yalnızca `:key` haline sadeleştirmenizi sağlar:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/12_for_in_short.dart](#)

```
for (var MapEntry(:key, value: count) in hist.entries) {
  print('$key occurred $count times');
}
```

Desenlerin kullanım alanları

[Önceki bölüm](#), desenlerin diğer Dart kod yapılarına *nasıl* uyduğunu anlatıyor. Örnek olarak iki değişkenin değerlerini [yer değiştirmek](#) ya da bir map'teki [anahtar-değer çiftlerini ayrıştırmak](#) gibi ilginç kullanımlar gördünüz. Bu bölüm daha da fazla kullanım alanını anlatarak şu sorulara cevap veriyor:

- Desenleri *ne zaman ve neden* kullanmak isteyebilirsiniz.
- Hangi tür sorunları çözerler.
- En çok hangi deyimisel kullanımlara uygundurlar.

Birden çok dönüş değerini ayrıştırma

Record'lar, tek bir fonksiyon çağrısından birden çok değeri toplayıp [döndürmeyi](#) sağlar. Desenler ise bir record'un alanlarını, fonksiyon çağrısıyla aynı satırda doğrudan yerel değişkenlere ayrıştırma yeteneği ekler.

Her record alanı için tek tek yeni yerel değişkenler tanımlamak yerine, yani şöyle yazmak yerine:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/13_destructure_multiple_returns_1.dart](#)

```
var info = userInfo(json);
var name = info.$1;
var age = info.$2;
```

Bir fonksiyonun döndürdüğü record'un alanlarını, bir [değişken tanımlı](#) ya da [atama deseni](#) ile, alt desen olarak da bir [record deseni](#) kullanarak yerel değişkenlere ayrıştırabilirsiniz:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/14_destructure_multiple_returns_2.dart](#)

```
var (name, age) = userInfo(json);
```

İsimli alanları olan bir record'u desenle ayrıştırmak için:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/15_destructure_multiple_returns_3.dart](#)

```
final (:name, :age) =
  getData(); // Örneğin, return (name: 'doug', age: 25);
```

Sınıf örneklerini ayrıştırma

[Nesne desenleri](#) isimli nesne tipleriyle eşleşir ve nesnenin sınıfının zaten sunduğu getter'ları kullanarak verilerini ayrıştırmanızı sağlar.

Bir sınıfın örneğini ayrıştırmak için isimli tipi, ardından ayrıştırılacak özellikleri parantez içinde yazın:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/16_destructure_class_instances.dart](#)

dart

```
final Foo myFoo = Foo(one: 'one', two: 2);
var Foo(:one, :two) = myFoo;
print('one $one, two $two');
```

Cebirsel veri tipleri

Nesne ayrıştırma ve switch case'leri, [cebirsel veri tipi \(algebraic data type\)](#) tarzında kod yazmaya elverişlidir. Bu yöntemi şu durumlarda kullanın:

- Birbiriyle ilişkili tiplerden oluşan bir aileniz varsa.
- Her tip için özel davranış gerektiren bir işleminiz varsa.
- Bu davranış tüm farklı tip tanımlarına dağıtmak yerine tek bir yerde toplamak istiyorsanız.

İşlemi her tip için bir örnek metodu olarak uygulamak yerine, işlemin çeşitlerini alt tipler üzerinde switch yapan tek bir fonksiyonda tutun:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/17_algebraic_datatypes.dart](#)

dart

```
sealed class Shape {}

class Square implements Shape {
  final double length;
  Square(this.length);
}

class Circle implements Shape {
  final double radius;
  Circle(this.radius);
}

double calculateArea(Shape shape) => switch (shape) {
  Square(length: var l) => l * l,
  Circle(radius: var r) => math.pi * r * r,
};
```

Gelen JSON'u doğrulama

[Map](#) ve [list](#) desenleri, JSON'dan ayrıştırılmış veriler gibi serileştirmesi çözülmüş (deserialized) verilerdeki anahtar-değer çiftlerini ayrıştırmak için çok uygundur:

[VS Code'da aç](#) [cods/2_patterns/1_patterns/18_json_1.dart](#)

dart

```
var data = {
  'user': ['Lily', 13],
};
var {'user': [name, age]} = data;
```

JSON verisinin beklediğiniz yapıda olduğunu biliyorsanız önceki örnek gerçekçidir. Ancak veri genellikle ağ gibi harici bir kaynaktan gelir. Yapısını doğrulamak için önce veriyi doğrulamanız gerekir.

Desenler olmadan doğrulama uzun ve zahmetlidir:

VS Code'da aç cods/2_patterns/1_patterns/19_json_2.dart

dart

```
if (data is Map<String, Object?> && data.containsKey('user')) {  
  var user = data['user'];  
  if (user is List<Object?> &&  
      user.length == 2 &&  
      user[0] is String &&  
      user[1] is int) {  
    var name = user[0] as String;  
    var age = user[1] as int;  
    print('User $name is $age years old.');  }  
}
```

Tek bir **case deseni** aynı doğrulamayı yapabilir. Tek case'ler en iyi **if-case** deyimleri olarak çalışır. Desenler JSON'u doğrulamak için daha bildirimsel ve çok daha kısa bir yöntem sunar:

VS Code'da aç cods/2_patterns/1_patterns/20_json_3.dart

dart

```
if (data case {'user': [String name, int age]}) {  
  print('User $name is $age years old.');}
```

Bu case deseni aynı anda şunları doğrular:

- `data` bir map'tir, çünkü devam etmek için önce dıştaki **map deseniyle** eşleşmelidir.
 - Ve bir map olduğu için `data` 'nın null olmadığını da doğrular.
- `data` bir `user` anahtarı içerir.
- `user` anahtarı iki değerli bir listeye eşleşir.
- Listedeki değerlerin tipleri `String` ve `int` 'tir.
- Değerleri tutacak yeni yerel değişkenler `name` ve `age` 'dir.

Desen tipleri

Dart'taki desen tipleri için başvuru kaynağı.

Bu sayfa farklı desen türleri için bir başvuru kaynağıdır. Desenlerin nasıl çalıştığına, Dart'ta nerelerde kullanılabildiğine ve yaygın kullanım alanlarına genel bir bakış için ana [Desenler](#) sayfasını ziyaret edin.

Desen önceliği

[Operatör önceliğine](#) benzer şekilde, desenlerin değerlendirilmesi de öncelik kurallarına uyar. Düşük öncelikli desenleri önce değerlendirmek için [parantezli desenler](#) kullanabilirsiniz.

Bu belge desen tiplerini artan öncelik sırasına göre listeler:

- [Mantıksal-veya](#) desenleri [mantıksal-ve](#) desenlerinden daha düşük önceliklidir, mantıksal-ve desenleri [ilişkisel](#) desenlerden daha düşük önceliklidir, ve bu böyle devam eder.
- Sonek (post-fix) tekli desenler ([cast](#), [null-check](#) ve [null-assert](#)) aynı öncelik düzeyini paylaşır.
- Geri kalan birincil desenler en yüksek önceliği paylaşır. Koleksiyon tipi ([record](#), [list](#) ve [map](#)) ve [Object](#) desenleri başka verileri kapsadığı için dış desenler olarak önce değerlendirilir.

Mantıksal-veya (logical-or)

```
subpattern1 || subpattern2
```

Bir mantıksal-veya deseni alt desenleri `||` ile ayırır ve dallardan herhangi biri eşleşirse eşleşir. Dallar soldan sağa değerlendirilir. Bir dal eşleştiğinde geri kalanlar değerlendirilmez.

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/01_logical_or.dart`

```
var isPrimary = switch (color) {
  Color.red || Color.yellow || Color.blue => true,
  _ => false,
};
```

dart

Bir mantıksal-veya desenindeki alt desenler değişken bağlayabilir, ancak dallar aynı değişken kümesini tanımlamalıdır, çünkü desen eşleştiğinde yalnızca bir dal değerlendirilecektir.

Mantıksal-ve (logical-and)

```
subpattern1 && subpattern2
```

`&&` ile ayrılmış bir desen çifti yalnızca iki alt desen de eşleşirse eşleşir. Sol dal eşleşmezse sağ dal değerlendirilmez.

Bir mantıksal-ve desenindeki alt desenler değişken bağlayabilir, ancak her alt desendeki değişkenler çakışmamalıdır, çünkü desen eşleşirse ikisi de bağlanacaktır:

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/02_logical_and.dart`

```
switch ((1, 2)) {
  // Hata, iki alt desen de 'b'yi bağlamaya çalışıyor.
  case (var a, var b) && (var b, var c): // ...
}
```

dart

İlişkisel (relational)

```
== expression
```

```
< expression
```

İlişkisel desenler, eşleşen değeri verilen bir sabitle şu eşitlik ya da ilişkisel operatörlerden herhangi birini kullanarak karşılaştırır: `==`, `!=`, `<`, `>`, `<=` ve `>=`.

Eşleşen değer üzerinde ilgili operatörü sabiti argüman olarak vererek çağırmak `true` döndürdüğünde desen eşleşir.

İlişkisel desenler, özellikle [mantıksal-ve deseniyle](#) birlikte kullanıldığında, sayısal aralıklarda eşleştirme yapmak için kullanışlıdır:

[VS Code'da aç](#) cods/2_patterns/2_pattern_types/03_relational.dart

dart

```
String asciiCharType(int char) {  
  const space = 32;  
  const zero = 48;  
  const nine = 57;  
  
  return switch (char) {  
    < space => 'control',  
    == space => 'space',  
    > space && < zero => 'punctuation',  
    >= zero && <= nine => 'digit',  
    - => '',  
  };  
}
```

Cast (tip dönüşümü)

foo as String

Bir cast deseni, değeri başka bir alt desene geçirmeden önce, ayrıştırmanın ortasına bir [tip dönüşümü](#) eklemenizi sağlar:

[VS Code'da aç](#) cods/2_patterns/2_pattern_types/04_cast.dart

dart

```
(num, Object) record = (1, 's');  
var (i as int, s as String) = record;
```

Değer belirtilen tipte değilse cast desenleri istisna [fırlatır](#). [null-assert deseni](#) gibi bu da ayrıştırılmış bir değer beklenen tipini zorla doğrulamanızı sağlar.

Null-check

subpattern?

Null-check desenleri önce değer null değilse eşleşir, ardından iç deseni aynı değerle eşleştirir. Eşleştirilen null olabilir değer null olamayan temel tipinde bir değişken bağlamanızı sağlarlar.

`null` değerleri istisna fırlatmadan eşleşme başarısızlığı olarak ele almak için null-check desenini kullanın.

[VS Code'da aç](#) cods/2_patterns/2_pattern_types/05_null_check.dart

dart

```
String? maybeString = 'nullable with base type String';  
switch (maybeString) {  
  case var s?:  
    // 's' burada null olamayan String tipindedir.  
}
```

Değer null *olduğunda* eşleştirmek için `null` [sabit desenini](#) kullanın.

Null-assert

subpattern!

Null-assert desenleri önce nesne null değilse eşleşir, sonra değer üzerinde eşleşir. Null olmayan değerlerin geçmesine izin verirler, ancak eşleşen değer null ise istisna [fırlatırlar](#).

`null` değerlerin sessizce eşleşme başarısızlığı olarak ele alınmamasını sağlamak için eşleştirme sırasında bir null-assert deseni kullanın:

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/06_null_assert_match.dart](#)

dart

```
List<String?> row = ['user', null];
switch (row) {
  case ['user', var name!]: // ...
    // 'name' burada null olamayan bir string'dir.
}
```

Değişken tanımlı desenlerinden `null` değerleri elemek için null-assert desenini kullanın:

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/07_null_assert_dec.dart](#)

dart

```
(int?, int?) position = (2, 3);

var (x!, y!) = position;
```

Değer null *olduğunda* eşleştirmek için `null` [sabit desenini](#) kullanın.

Sabit (constant)

`123`, `null`, `'string'`, `math.pi`, `SomeClass.constant`, `const Thing(1, 2)`, `const (1 + 2)`

Sabit desenleri, değer sabite eşit olduğunda eşleşir:

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/08_constant.dart](#)

dart

```
switch (number) {
  // 1 == number ise eşleşir.
  case 1: // ...
}
```

Basit literal'leri ve isimli sabitlere yapılan başvuruları doğrudan sabit deseni olarak kullanabilirsiniz:

- Sayı literal'leri (`123`, `45.56`)
- Boolean literal'leri (`true`)
- String literal'leri (`'string'`)
- İsimli sabitler (`someConstant`, `math.pi`, `double.infinity`)
- Sabit yapıcılar (`const Point(0, 0)`)
- Sabit koleksiyon literal'leri (`const []`, `const {1, 2}`)

Daha karmaşık sabit ifadeler paranteze alınmalı ve önüne `const` eklenmelidir (`const (1 + 2)`):

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/09_complex_constant.dart](#)

dart

```
// List ya da map deseni:
case [a, b]: // ...

// List ya da map literal'i:
case const [a, b]: // ...
```

Değişken (variable)

`var bar`, `String str`, `final int _`

Değişken desenleri, eşleşmiş ya da ayrıştırılmış değerlere yeni değişkenler bağlar. Genellikle ayrıştırılmış bir değeri yakalamak için bir [ayırıştırma deseninin](#) parçası olarak bulunurlar.

Değişkenler, yalnızca desen eşleştğinde ulaşılabilen kod bölgesinde kapsam içindedir.

VS Code'da aç cods/2_patterns/2_pattern_types/10_variable.dart

dart

```
switch ((1, 2)) {  
  // 'var a' ve 'var b', sırasıyla 1 ve 2'ye bağlanan değişken desenleridir.  
  case (var a, var b): // ...  
    // 'a' ve 'b', case gövdesinde kapsam içindedir.  
}
```

Tipli bir değişken deseni yalnızca eşleşen değer tanımlanan tipteyse eşleşir, aksi halde başarısız olur:

VS Code'da aç cods/2_patterns/2_pattern_types/11_variable_typed.dart

dart

```
switch ((1, 2)) {  
  // Eşleşmez.  
  case (int a, String b): // ...  
}
```

Bir **joker deseni** değişken deseni olarak kullanabilirsiniz.

Tanımlayıcı (identifier)

foo, _

Tanımlayıcı desenleri, bulundukları bağlama göre bir **sabit deseni** ya da bir **değişken deseni** gibi davranır:

- Tanım** bağlamı: Tanımlayıcı adıyla yeni bir değişken tanımlar: `var (a, b) = (1, 2);`
- Atama** bağlamı: Tanımlayıcı adıyla var olan değişkene atama yapar: `(a, b) = (3, 4);`
- Eşleştirme** bağlamı: İsimli bir sabit deseni olarak ele alınır (adı `_` değilse):

VS Code'da aç cods/2_patterns/2_pattern_types/12_match_context.dart

dart

```
const c = 1;  
switch (2) {  
  case c:  
    print('match $c');  
  default:  
    print('no match'); // "no match" yazdırır.  
}
```

- Herhangi bir bağlamda **joker** tanımlayıcı: Her değerle eşleşir ve onu atar: `case (_, var y, _): print('The middle element is $y');`

Parantezli (parenthesized)

(subpattern)

Parantezli ifadelerde olduğu gibi, bir desendeki parantezler **desen önceliğini** kontrol etmenizi ve daha yüksek öncelikli bir desen beklenen yere daha düşük öncelikli bir desen yerleştirmenizi sağlar.

Örneğin `x`, `y` ve `z` boolean sabitlerinin sırasıyla `true`, `true` ve `false` olduğunu düşünün. Aşağıdaki örnek boolean ifade değerlendirmesine benzese de aslında desen eşleştirmesi yapar.

VS Code'da aç cods/2_patterns/2_pattern_types/13_parens.dart

dart

```
// ...  
x || y => 'matches true',  
x || y && z => 'matches true',  
x || (y && z) => 'matches true',  
// 'x || y && z', 'x || (y && z)' ile aynı şeydir.  
(x || y) && z => 'matches nothing',  
// ...
```

Dart deseni soldan sağa doğru eşleştirmeye başlar.

1. İlk desen `true` ile eşleşir, çünkü `x` `true` ile eşleşir.
2. İkinci desen `true` ile eşleşir, çünkü `x` `true` ile eşleşir.
3. Üçüncü desen `true` ile eşleşir, çünkü `x` `true` ile eşleşir.
4. Dördüncü desen `(x || y) && z` hiçbir şeyle eşleşmez.
 - `x` `true` ile eşleştiği için Dart `y` 'yi eşleştirmeyi denemez.
 - `(x || y)` `true` ile eşleşse de `z` `true` ile eşleşmez.
 - Bu yüzden `(x || y) && z` deseni `true` ile eşleşmez.
 - `(x || y)` alt deseni `false` ile eşleşmez, bu yüzden Dart `z` 'yi eşleştirmeyi denemez.
 - Bu yüzden `(x || y) && z` deseni `false` ile eşleşmez.
 - Sonuç olarak `(x || y) && z` hiçbir şeyle eşleşmez.

List

`[subpattern1, subpattern2]`

Bir list deseni `List` 'i uygulayan değerlerle eşleşir, ardından alt desenlerini listenin elemanlarıyla özyinelemeli olarak eşleştirerek onları konumlarına göre ayırır:

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/14_list_pattern.dart`

dart

```
const a = 'a';
const b = 'b';
switch (obj) {
  // [a, b] list deseni, obj iki alanlı bir liste ise önce obj ile eşleşir,
  // sonra alanları 'a' ve 'b' sabit alt desenleriyle eşleşirse.
  case [a, b]:
    print('$a, $b');
}
```

List desenleri, desendeki eleman sayısının listenin tamamıyla eşleşmesini gerektirir. Ancak listedeki herhangi sayıda elemanı karşılamak için yer tutucu olarak bir [rest elemanı](#) kullanabilirsiniz.

Rest elemanı

List desenleri, keyfi uzunluktaki listelerle eşleşmeye olanak tanıyan *bir* rest elemanı `(...)` içerebilir.

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/15_rest.dart`

dart

```
var [a, b, ..., c, d] = [1, 2, 3, 4, 5, 6, 7];
// "1 2 6 7" yazdırır.
print('$a $b $c $d');
```

Bir rest elemanının, listedeki diğer alt desenlerle eşleşmeyen elemanları yeni bir listede toplayan bir alt deseni de olabilir:

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/16_rest_sub.dart`

dart

```
var [a, b, ...rest, c, d] = [1, 2, 3, 4, 5, 6, 7];
// "1 2 [3, 4, 5] 6 7" yazdırır.
print('$a $b $rest $c $d');
```

Map

`{"key": subpattern1, someConst: subpattern2}`

Map desenleri `Map` 'i uygulayan değerlerle eşleşir, ardından alt desenlerini map'in anahtarlarıyla özyinelemeli olarak eşleştirerek onları ayırır.

Map desenleri desenin map'in tamamıyla eşleşmesini gerektirmez. Bir map deseni, map'in içerdiği ama desenle eşleşmeyen anahtarları yok sayar. Map'te olmayan bir anahtar eşleştirmeye çalışmak bir `StateError` fırlatır:

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/17_map_error.dart`

dart

```
final {'foo': int? foo} = {};
```

Record

```
(subpattern1, subpattern2)
```

```
(x: subpattern1, y: subpattern2)
```

Record desenleri bir [record](#) nesnesiyle eşleşir ve alanlarını ayırır. Değer, desenle aynı [şekle](#) sahip bir record değilse eşleşme başarısız olur. Aksi halde alan alt desenleri record'daki karşılık gelen alanlarla eşleştirilir.

Record desenleri, desenin record'un tamamıyla eşleşmesini gerektirir. *İsimli* alanları olan bir record'u desenle ayırtmak için alan adlarını desene ekleyin:

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/18_record.dart](#)

```
var (myString: foo, myNumber: bar) = (myString: 'string', myNumber: 1);
```

dart

Getter adı yazılmayabilir ve alan alt desenindeki [değişken deseninden](#) ya da [tanımlayıcı deseninden](#) çıkarılabilir. Aşağıdaki desen çiftlerinin her biri birbirine denktir:

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/19_record_getter.dart](#)

```
// Değişken alt desenli record deseni:
var (untyped: untyped, typed: int typed) = record;
var (:untyped, :int typed) = record;

switch (record) {
  case (untyped: var untyped, typed: int typed): // ...
  case (:var untyped, :int typed): // ...
}

// null-check ve null-assert alt desenli record deseni:
switch (record) {
  case (checked: var checked?, asserted: var asserted!): // ...
  case (:var checked?, :var asserted!): // ...
}

// cast alt desenli record deseni:
var (untyped: untyped as int, typed: typed as String) = record;
var (:untyped as int, :typed as String) = record;
```

dart

Object (nesne)

```
SomeClass(x: subpattern1, y: subpattern2)
```

Nesne desenleri, nesnenin özelliklerindeki getter'ları kullanarak veriyi ayırtmak için eşleşen değeri verilen bir isimli tuple karşılaştırır. Değer aynı tipte değilse [çürütülürler \(refuted\)](#), yani eşleşmezler.

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/20_object.dart](#)

```
switch (shape) {
  // shape Rect tipindeyse eşleşir, sonra Rect'in özellikleriyle karşılaştırılır.
  case Rect(width: var w, height: var h): // ...
}
```

dart

Getter adı yazılmayabilir ve alan alt desenindeki [değişken deseninden](#) ya da [tanımlayıcı deseninden](#) çıkarılabilir:

[VS Code'da aç](#) [cods/2_patterns/2_pattern_types/21_object_getter.dart](#)

```
// Yeni x ve y değişkenlerini Point'in x ve y özelliklerinin değerlerine bağlar.
var Point(:x, :y) = Point(1, 2);
```

dart

Nesne desenleri desenin nesnenin tamamıyla eşleşmesini gerektirmez. Bir nesnenin desenin ayırtmadığı ek alanları olsa bile eşleşebilir.

Joker (wildcard)

–

`_` adlı bir desen jokerdir; hiçbir değişkene bağlama ya da atama yapmayan bir [değişken deseni](#) ya da [tanımlayıcı desenidir](#).

Sonraki konumsal değerleri ayrıştırabilmek için bir alt desene ihtiyaç duyduğunuz yerlerde yer tutucu olarak kullanışlıdır:

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/22_wildcard.dart`

```
var list = [1, 2, 3];  
var [_ , two, _] = list;
```

dart

Tip belirtilmi bir joker adı, bir değerini tipini test etmek ama değeri bir isme bağlamamak istediğinizde kullanışlıdır:

[VS Code'da aç](#) `cods/2_patterns/2_pattern_types/23_wildcard_typed.dart`

```
switch (record) {  
  case (int _, String _):  
    print('First field is int and second is String.');
```

dart

Döngüler

Dart kodunuzun akışını kontrol etmek için döngüleri nasıl kullanacağınızı öğrenin.

Bu sayfa, Dart kodunuzun akışını döngüler ve onları destekleyen deyimlerle nasıl kontrol edebileceğinizi gösterir:

- `for` döngüleri
- `while` ve `do while` döngüleri
- `break` ve `continue`

Dart'ta kontrol akışını şunlarla da yönetebilirsiniz:

- `if` ve `switch` gibi [dallanmalar](#)
- `try`, `catch` ve `throw` gibi [istisnalar](#)

For döngüleri

Standart `for` döngüsüyle yineleme yapabilirsiniz. Örneğin:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/01_for.dart`

dart

```
var message = StringBuffer('Dart is fun');
for (var i = 0; i < 5; i++) {
  message.write('!');
}
```

Dart'ın `for` döngüleri içindeki closure'lar indeksin *değerini* yakalar. Bu, JavaScript'te sık karşılaşılan bir tuzaktan kaçınmayı sağlar. Örneğin şunu düşünün:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/02_for_and_closures.dart`

dart

```
var callbacks = [];
for (var i = 0; i < 2; i++) {
  callbacks.add(() => print(i));
}

for (final c in callbacks) {
  c();
}
```

Çıktı beklendiği gibi önce `0`, sonra `1` olur. Buna karşılık aynı örnek JavaScript'te önce `2`, sonra yine `2` yazdırırdı.

`List` ya da `Set` gibi bir [Iterable](#) tip üzerinde dolaşırken bazen o anki yineleme sayacını bilmeniz gerekmeyebilir. Bu durumda daha temiz bir kod için `for-in` döngüsünü kullanın:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/03_collection.dart`

dart

```
for (var candidate in candidates) {
  candidate.interview();
}
```

Önceki örnek döngüde `candidate` döngü gövdesi içinde tanımlanır ve her seferinde `candidates` 'teki bir değere başvuracak şekilde ayarlanır. `candidate` yerel bir [değişkendir](#). Döngü gövdesi içinde `candidate` 'e yeniden değer atamak yalnızca o yinelemedeki yerel değişkeni değiştirir, orijinal `candidates` iterable'ını değiştirmez.

Iterable'dan elde edilen değerleri işlemek için bir `for-in` döngüsünde [desen](#) de kullanabilirsiniz:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/04_collection_for_pattern.dart`

dart

```
for (final Candidate(:name, :yearsExperience) in candidates) {
  print('$name has $yearsExperience of experience.');
}
```

İpucu

`for-in` kullanımını pratik etmek için [Iterable collections eğitimini](#) takip edin.

Iterable sınıflarının başka bir seçenek olarak bir `forEach()` metodu da vardır:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/05_for_each.dart`

dart

```
var collection = [1, 2, 3];
collection.forEach(print); // 1 2 3
```

While ve do-while

Bir `while` döngüsü koşulu döngüden önce değerlendirir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/06_while.dart`

dart

```
while (!isDone()) {
  doSomething();
}
```

Bir `do-while` döngüsü koşulu döngüden *sonra* değerlendirir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/07_do_while.dart`

dart

```
do {
  printLine();
} while (!atEndOfPage());
```

Break ve continue

Döngüyü durdurmak için `break` kullanın:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/08_while_break.dart`

dart

```
while (true) {
  if (shutdownRequested()) break;
  processIncomingRequests();
}
```

Bir sonraki yinelemeye geçmek için `continue` kullanın:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/09_for_continue.dart`

dart

```
for (int i = 0; i < candidates.length; i++) {
  var candidate = candidates[i];
  if (candidate.yearsExperience < 5) {
    continue;
  }
  candidate.interview();
}
```

List ya da set gibi bir `Iterable` kullanıyorsanız önceki örneği farklı bir şekilde yazabilirsiniz:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/10_where.dart`

dart

```
candidates
  .where((c) => c.yearsExperience >= 5)
  .forEach((c) => c.interview());
```

Etiketler (labels)

Etiket, bir deyimin önüne yerleştirilerek *etiketli bir deyim* oluşturabileceğiniz, ardından iki nokta gelen bir tanımlayıcıdır (etiketAdi:). Döngüler ve switch case'leri sıklıkla etiketli deyim olarak kullanılır. Etiketli bir deyime daha sonra bir `break` ya da `continue` deyiminde şu şekilde başvurulabilir:

- `break` `labelName`; Etiketli deyimin çalışmasını sonlandırır. İç içe bir döngünün içindeyken belirli bir dış döngüden çıkmak için kullanışlıdır.
- `continue` `labelName`; Etiketli döngü deyiminin o anki yinemesinin geri kalanını atlar ve bir sonraki yinemeyle devam eder.

Etiketler kontrol akışını yönetmek için kullanılır. Çoğunlukla döngüler ve switch case'leriyle birlikte kullanılırlar ve varsayılan olarak en içteki döngüyü etkilemek yerine hangi deyimden çıkılacağını ya da hangisine devam edileceğini belirtmenizi sağlarlar.

For döngüsünde `break` ile etiket kullanımı

Aşağıdaki kod, `outerLoop` adlı bir etiketin bir `for` döngüsünde `break` deyimiyle kullanımını gösterir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/11_label_for_loop_break.dart`

```
outerLoop:
  for (var i = 1; i <= 3; i++) {
    for (var j = 1; j <= 3; j++) {
      print('i = $i, j = $j');
      if (i == 2 && j == 2) {
        break outerLoop;
      }
    }
  }
  print('outerLoop exited');
```

Önceki örnekte `i == 2` ve `j == 2` olduğunda `break outerLoop;` deyimi hem iç hem de dış döngüyü durdurur. Sonuç olarak çıktı şöyledir:

```
i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 2, j = 2
outerLoop exited
```

For döngüsünde `continue` ile etiket kullanımı

Aşağıdaki kod, `outerLoop` adlı bir etiketin bir `for` döngüsünde `continue` deyimiyle kullanımını gösterir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/12_label_for_loop_continue.dart`

```
outerLoop:
  for (var i = 1; i <= 3; i++) {
    for (var j = 1; j <= 3; j++) {
      if (i == 2 && j == 2) {
        continue outerLoop;
      }
      print('i = $i, j = $j');
    }
  }
```

Önceki örnekte `i == 2` ve `j == 2` olduğunda `continue outerLoop;` `i = 2` için kalan yinelemeleri atlar ve `i = 3` 'e geçer. Sonuç olarak çıktı şöyledir:

```
i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 3, j = 1
i = 3, j = 2
i = 3, j = 3
```

While döngüsünde `break` ile etiket kullanımı

Aşağıdaki kod, `outerLoop` adlı bir etiketin bir `while` döngüsünde `break` deyiimiyle kullanımını gösterir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/13_label_while_loop_break.dart`

dart

```
var i = 1;

outerLoop:
while (i <= 3) {
  var j = 1;
  while (j <= 3) {
    print('i = $i, j = $j');
    if (i == 2 && j == 2) {
      break outerLoop;
    }
    j++;
  }
  i++;
}
print('outerLoop exited');
```

Önceki örnekte program, `i == 2` ve `j == 2` olduğunda hem iç hem de dış `while` döngüsünden çıkar. Sonuç olarak çıktı şöyledir:

```
i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 2, j = 2
outerLoop exited
```

While döngüsünde `continue` ile etiket kullanımı

Aşağıdaki kod, `outerLoop` adlı bir etiketin bir `while` döngüsünde `continue` deyimiiyle kullanımını gösterir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/14_label_while_loop_continue.dart`

dart

```
var i = 1;

outerLoop:
while (i <= 3) {
  var j = 1;
  while (j <= 3) {
    if (i == 2 && j == 2) {
      i++;
      continue outerLoop;
    }
    print('i = $i, j = $j');
    j++;
  }
  i++;
}
```

Önceki örnekte `i = 2` ve `j = 2` için yinleme atlanır ve döngü doğrudan `i = 3`'e geçer. Sonuç olarak çıktı şöyledir:

```
i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 3, j = 1
i = 3, j = 2
i = 3, j = 3
```

Do-while döngüsünde `break` ile etiket kullanımı

Aşağıdaki kod, `outerLoop` adlı bir etiketin bir `do while` döngüsünde `break` deyimiiyle kullanımını gösterir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/15_label_do_while_loop_break.dart`

dart

```
var i = 1;

outerLoop:
do {
  var j = 1;
  do {
    print('i = $i, j = $j');
    if (i == 2 && j == 2) {
      break outerLoop;
    }
    j++;
  } while (j <= 3);
  i++;
} while (i <= 3);

print('outerLoop exited');
```

Önceki örnekte program, `i == 2` ve `j == 2` olduğunda hem iç hem de dış döngüden çıkar. Sonuç olarak çıktı şöyledir:

```
i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 2, j = 2
outerLoop exited
```

Do-while döngüsünde `continue` ile etiket kullanımı

Aşağıdaki kod, `outerLoop` adlı bir etiketin bir `do while` döngüsünde `continue` deyimiyile kullanımını gösterir:

[VS Code'da aç](#) `cods/3_control_flow/1_loops/16_label_do_while_loop_continue.dart`

```
var i = 1;

outerLoop:
do {
  var j = 1;
  do {
    if (i == 2 && j == 2) {
      i++;
      continue outerLoop;
    }
    print('i = $i, j = $j');
    j++;
  } while (j <= 3);
  i++;
} while (i <= 3);
```

Önceki örnekte döngü `i = 2` ve `j = 2` 'yi atlar ve doğrudan `i = 3` 'e geçer. Sonuç olarak çıktı şöyledir:

```
i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 3, j = 1
i = 3, j = 2
i = 3, j = 3
```

Dallanmalar

Dart kodunuzun akışını kontrol etmek için dallanmaları nasıl kullanacağınızı öğrenin.

Bu sayfa, Dart kodunuzun akışını dallanmalarla nasıl kontrol edebileceğinizi gösterir:

- `if` deyimleri ve elemanları
- `if-case` deyimleri ve elemanları
- `switch` deyimleri ve ifadeleri

Dart'ta kontrol akışını şunlarla da yönetebilirsiniz:

- `for` ve `while` gibi [döngüler](#)
- `try`, `catch` ve `throw` gibi [istisnalar](#)

If

Dart, isteğe bağlı `else` cümleleriyle `if` deyimlerini destekler. `if` 'ten sonra parantez içindeki koşul, [boolean](#) sonuç veren bir ifade olmalıdır:

[VS Code'da aç](#) `cods/3_control_flow/2_branches/01_if_else.dart`

```
if (isRaining()) {
  you.bringRainCoat();
} else if (isSnowing()) {
  you.wearJacket();
} else {
  car.putTopDown();
}
```

dart

`if` 'i bir ifade bağlamında nasıl kullanacağınızı öğrenmek için [Conditional expressions](#) bölümüne göz atın.

If-case

Dart'taki `if` deyimleri, ardından bir [desen](#) gelen `case` cümlelerini destekler:

[VS Code'da aç](#) `cods/3_control_flow/2_branches/02_if_case.dart`

```
if (pair case [int x, int y]) return Point(x, y);
```

dart

Desen değerle eşleşirse dal, desenin tanımladığı değişkenler kapsam içindeyken çalışır.

Önceki örnekte `[int x, int y]` list deseni `pair` değeriyle eşleşir, bu yüzden `return Point(x, y)` dalı desenin tanımladığı `x` ve `y` değişkenleriyle çalışır.

Aksi halde, varsa çalıştırılmak üzere kontrol akışı `else` dalına geçer:

[VS Code'da aç](#) `cods/3_control_flow/2_branches/03_if_case_else.dart`

```
if (pair case [int x, int y]) {
  print('Was coordinate array $x,$y');
} else {
  throw FormatException('Invalid coordinates.');
```

dart

If-case deyimini, *tek* bir desenle eşleştirme ve [ayrıştırma](#) yapmanın bir yoludur. Bir değeri *birden çok* desenle test etmek için [switch](#) kullanın.

↗ Sürüm notu

`if` deyimlerindeki `case` cümleleri en az 3.0 [dil sürümü](#) gerektirir.

Switch deyimleri

Bir `switch` deyimi, bir deęer ifadesini bir dizi case'e karşı deęerlendirir. Her `case` cümlesi, deęerin eşleřtirileceęi bir [desendir](#). Bir case için [her tür deseni](#) kullanabilirsiniz.

Deęer bir case'in deseniyle eşleřtięinde case gövdesi çalışır. Boř olmayan `case` cümleleri tamamlandıktan sonra switch'in sonuna atlar. `break` deyimi gerektirmezler. Boř olmayan bir `case` cümlesini bitirmenin dięer geęerli yolları `continue`, `throw` ya da `return` deyimidir.

Hiębir `case` cümlesi eşleřmedięinde kod çalıştırmak için `default` ya da `joker _` cümlesi kullanın:

[VS Code'da aç](#) `cods/3_control_flow/2_branches/04_switch.dart`

dart

```
var command = 'OPEN';
switch (command) {
  case 'CLOSED':
    executeClosed();
  case 'PENDING':
    executePending();
  case 'APPROVED':
    executeApproved();
  case 'DENIED':
    executeDenied();
  case 'OPEN':
    executeOpen();
  default:
    executeUnknown();
}
```

Boř case'ler bir sonraki case'e düşer (fall through); böylece case'ler aynı gövdeyi paylaşabilir. Bir sonrakine düşmeyen boř bir case için gövde olarak `break` kullanın. Sıralı olmayan bir düşüş için bir `continue` deyimi ve bir etiket kullanabilirsiniz:

[VS Code'da aç](#) `cods/3_control_flow/2_branches/05_switch_empty.dart`

dart

```
switch (command) {
  case 'OPEN':
    executeOpen();
    continue newCase; // newCase etiketinden çalışmaya devam eder.

  case 'DENIED': // Boř case bir sonrakine düşer (fall through).
  case 'CLOSED':
    executeClosed(); // Hem DENIED hem de CLOSED için çalışır,

  newCase:
  case 'PENDING':
    executeNowClosed(); // Hem OPEN hem de PENDING için çalışır.
}
```

Case'lerin aynı gövdeyi ya da koruma ifadesini paylaşmasını sağlamak için [mantıksal-veya desenlerini](#) kullanabilirsiniz. Desenler ve case cümleleri hakkında daha fazla bilgi için desen dokümantasyonundaki [Switch deyimleri ve ifadeleri](#) bölümüne göz atın.

Switch ifadeleri

Bir `switch` ifadesi, hangi case eşleřirse onun ifade gövdesine göre bir deęer üretir. Switch ifadesini Dart'ın ifadelere izin verdięi her yerde kullanabilirsiniz; bir ifade deyiminin *bařı harię*. Örneęin:

[VS Code'da aç](#) `cods/3_control_flow/2_branches/06_ornek.dart`

dart

```
var x = switch (y) { ... };

print(switch (x) { ... });

return switch (x) { ... };
```

Bir ifade deyiminin başında switch kullanmak istiyorsanız bir [switch deyimi](#) kullanın.

Switch ifadeleri, şuna benzer bir switch *deyimini*:

[VS Code'da aç](#) cods/3_control_flow/2_branches/07_switch_stmt.dart

```
// Burada slash, star, comma, semicolon vb. sabit değişkenlerdir...
switch (charCode) {
  case slash || star || plus || minus: // Mantıksal-veya deseni
    token = operator(charCode);
  case comma || semicolon: // Mantıksal-veya deseni
    token = punctuation(charCode);
  case >= digit0 && <= digit9: // İlişkisel ve mantıksal-ve desenleri
    token = number();
  default:
    throw FormatException('Invalid');
}
```

Şuna benzer bir *ifadeye* dönüştürmenizi sağlar:

[VS Code'da aç](#) cods/3_control_flow/2_branches/08_switch_exp.dart

```
token = switch (charCode) {
  slash || star || plus || minus => operator(charCode),
  comma || semicolon => punctuation(charCode),
  >= digit0 && <= digit9 => number(),
  _ => throw FormatException('Invalid'),
};
```

Bir `switch` ifadesinin sözdizimi `switch` deyiminin sözdiziminden farklıdır:

- Case'ler `case` anahtar kelimesiyle *başlamaz*.
- Case gövdesi bir dizi deyim yerine tek bir ifadedir.
- Her case'in bir gövdesi olmalıdır; boş case'ler için örtük düşüş (fallthrough) yoktur.
- Case desenleri gövdelerinden `:` yerine `=>` ile ayrılır.
- Case'ler `,` ile ayrılır (sonda isteğe bağlı bir `,` kullanılabilir).
- Varsayılan case'ler hem `default` hem de `_` yerine *yalnızca* `_` kullanabilir.

[Sürüm notu](#)

Switch ifadeleri en az 3.0 [dil sürümü](#) gerektirir.

Kapsayıcılık (exhaustiveness) kontrolü

Kapsayıcılık kontrolü, bir değerin switch'e girip hiçbir case ile eşleşmemesi mümkünse derleme zamanı hatası bildiren bir özelliktir.

[VS Code'da aç](#) cods/3_control_flow/2_branches/09_exh_bool.dart

```
// bool? üzerinde kapsayıcı olmayan switch, null olasılığını karşılayan case eksik:
switch (nullableBool) {
  case true:
    print('yes');
  case false:
    print('no');
}
```

Varsayılan bir case (`default` ya da `_`) switch'ten geçebilecek tüm olası değerleri kapsar. Bu, herhangi bir tip üzerindeki switch'i kapsayıcı hale getirir.

Enum'lar ve mühürlü (sealed) tipler switch'ler için özellikle kullanışlıdır, çünkü varsayılan bir case olmasa bile olası değerleri bilinir ve tamamen sayılabilir. Bir sınıfın alt tipleri üzerinde switch yaparken kapsayıcılık kontrolünü etkinleştirmek için o sınıfta `sealed` niteleyicisini kullanın:

[VS Code'da aç](#) `cods/3_control_flow/2_branches/10_algebraic_datatypes.dart`

dart

```
sealed class Shape {}

class Square implements Shape {
  final double length;
  Square(this.length);
}

class Circle implements Shape {
  final double radius;
  Circle(this.radius);
}

double calculateArea(Shape shape) => switch (shape) {
  Square(length: var l) => l * l,
  Circle(radius: var r) => math.pi * r * r,
};
```

Biri `Shape`'in yeni bir alt sınıfını eklerse bu `switch` ifadesi eksik kalır. Kapsayıcılık kontrolü sizi eksik alt tip hakkında bilgilendirir. Bu, Dart'ı bir ölçüde [fonksiyonel cebirsel veri tipi tarzında](#) kullanmanızı sağlar.

Koruma ifadesi (guard clause)

Bir `case` cümlesinden sonra isteğe bağlı bir koruma ifadesi koymak için `when` anahtar kelimesini kullanın. Bir koruma ifadesi `if case`'in, ayrıca hem `switch` deyimlerinin hem de ifadelerinin ardından gelebilir.

[VS Code'da aç](#) `cods/3_control_flow/2_branches/11_ornek.dart`

dart

```
// Switch deymi:
switch (something) {
  case somePattern when some || boolean || expression:
    //          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Koruma ifadesi (guard clause).
    body;
}

// Switch ifadesi:
var value = switch (something) {
  somePattern when some || boolean || expression => body,
  //          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Koruma ifadesi (guard clause).
}

// If-case deymi:
if (something case somePattern when some || boolean || expression) {
  //          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Koruma ifadesi (guard clause).
  body;
}
```

Koruma ifadeleri, eşleştirmeden *sonra* keyfi bir boolean ifadeyi değerlendirir. Bu, bir case gövdesinin çalışıp çalışmayacağına dair ek kısıtlamalar eklemenizi sağlar. Koruma ifadesi false sonuç verdiğinde çalışma, tüm switch'ten çıkmak yerine bir sonraki case'e geçer.

Hata yönetimi

Dart'ta hataları ve istisnaları ele almayı öğrenin.

İstisnalar (exceptions)

Dart kodunuz istisna fırlatabilir ve yakalayabilir. İstisnalar, beklenmedik bir şey olduğunu belirten hatalardır. İstisna yakalanmazsa istisnayı fırlatan **isolate** askıya alınır ve genellikle isolate ile programı sonlandırılır.

Java'nın aksine Dart'taki tüm istisnalar denetlenmeyen (unchecked) istisnalardır. Metotlar hangi istisnaları fırlatabileceklerini bildirmez ve sizin de herhangi bir istisnayı yakalama zorunluluğunuz yoktur.

Dart, **Exception** ve **Error** tiplerini ve önceden tanımlanmış çok sayıda alt tipini sunar. Elbette kendi istisnalarınızı da tanımlayabilirsiniz. Bununla birlikte Dart programları yalnızca Exception ve Error nesnelerini değil, null olmayan herhangi bir nesneyi istisna olarak fırlatabilir.

Throw

Bir istisna fırlatma, yani istisna *oluşturma* örneği:

[VS Code'da aç](#) cods/3_control_flow/3_error_handling/01_throw_formatexception.dart

```
throw FormatException('Expected at least 1 section');
```

dart

Keyfi nesneler de fırlatabilirsiniz:

[VS Code'da aç](#) cods/3_control_flow/3_error_handling/02_out_of_llamas.dart

```
throw 'Out of llamas!';
```

dart

Not

Üretim kalitesindeki kod genellikle **Error** ya da **Exception** 'ı uygulayan tipler fırlatır.

İstisna fırlatmak bir ifade olduğu için istisnaları => deyimlerinde ve ifadelere izin verilen diğer her yerde fırlatabilirsiniz:

[VS Code'da aç](#) cods/3_control_flow/3_error_handling/03_throw_is_an_expression.dart

```
void distanceTo(Point other) => throw UnimplementedError();
```

dart

Catch

Bir istisnayı yakalamak (catch), istisnanın yayılmasını durdurur (istisnayı yeniden fırlatmadığınız sürece). Bir istisnayı yakalamak size onu ele alma fırsatı verir:

[VS Code'da aç](#) cods/3_control_flow/3_error_handling/04_try.dart

```
try {
  breedMoreLlamas();
} on OutOfLlamasException {
  buyMoreLlamas();
}
```

dart

Birden fazla türde istisna fırlatabilen kodu ele almak için birden çok catch cümlesi belirtebilirsiniz. Fırlatılan nesnenin tipiyle eşleşen ilk catch cümlesi istisnayı ele alır. catch cümlesi bir tip belirtmiyorsa o cümle fırlatılan her tipteki nesneyi ele alabilir:

[VS Code'da aç](#) [cods/3_control_flow/3_error_handling/05_try_catch.dart](#)

dart

```
try {
  breedMoreLlamas();
} on OutOfLlamasException {
  // Belirli bir istisna
  buyMoreLlamas();
} on Exception catch (e) {
  // İstisna olan diğer her şey
  print('Unknown exception: $e');
} catch (e) {
  // Tip belirtilmemiş, hepsini yakalar
  print('Something really unknown: $e');
}
```

Yukarıdaki kodun gösterdiği gibi `on`, `catch` ya da ikisini birden kullanabilirsiniz. İstisna tipini belirtmeniz gerektiğinde `on` kullanın. İstisna işleyicinizin istisna nesnesine ihtiyacı olduğunda `catch` kullanın.

`catch()`'e bir ya da iki parametre verebilirsiniz. Birincisi fırlatılan istisnadır, ikincisi ise yığın izidir (bir `StackTrace` nesnesi).

[VS Code'da aç](#) [cods/3_control_flow/3_error_handling/06_try_catch_2.dart](#)

dart

```
try {
  // ...
} on Exception catch (e) {
  print('Exception details:\n $e');
} catch (e, s) {
  print('Exception details:\n $e');
  print('Stack trace:\n $s');
}
```

Bir istisnayı kısmen ele alıp yayılmaya devam etmesine izin vermek için `rethrow` anahtar kelimesini kullanın.

[VS Code'da aç](#) [cods/3_control_flow/3_error_handling/07_rethrow.dart](#)

dart

```
void misbehave() {
  try {
    dynamic foo = true;
    print(foo++); // Çalışma zamanı hatası
  } catch (e) {
    print('misbehave() partially handled ${e.runtimeType}.');
    rethrow; // Çağırانların istisnayı görmesine izin ver.
  }
}

void main() {
  try {
    misbehave();
  } catch (e) {
    print('main() finished handling ${e.runtimeType}.');
  }
}
```

Finally

İstisna fırlatılsa da fırlatılmasa da bir kodun çalışmasını sağlamak için `finally` cümlesi kullanın. İstisnaya eşleşen bir `catch` cümlesi yoksa istisna `finally` cümlesi çalıştıktan sonra yayılır:

[VS Code'da aç](#) `cods/3_control_flow/3_error_handling/08_finally.dart`

```
try {  
  breedMoreLlamas();  
} finally {  
  // İstisna fırlatılsa bile her zaman temizlik yap.  
  cleanLlamaStalls();  
}
```

`finally` cümlesi, eşleşen `catch` cümlelerinden sonra çalışır:

[VS Code'da aç](#) `cods/3_control_flow/3_error_handling/09_try_catch_finally.dart`

```
try {  
  breedMoreLlamas();  
} catch (e) {  
  print('Error: $e'); // Önce istisnayı ele al.  
} finally {  
  cleanLlamaStalls(); // Sonra temizlik yap.  
}
```

Daha fazla bilgi için [çekirdek kütüphanenin istisna dokümantasyonuna](#) göz atın.

Assert

Geliştirme sırasında, bir boolean koşul false ise normal çalışmayı kesmek için `assert(<koşul>, <isteğeBağlıMesaj>);` şeklindeki assert deyimini kullanın.

[VS Code'da aç](#) `cods/3_control_flow/3_error_handling/10_assert.dart`

```
// Değişkenin null olmayan bir değeri olduğundan emin ol.  
assert(text != null);  
  
// Değerin 100'den küçük olduğundan emin ol.  
assert(number < 100);  
  
// Bunun bir https URL'si olduğundan emin ol.  
assert(urlString.startsWith('https'));
```

Bir doğrulamaya (assertion) mesaj eklemek için `assert` 'e ikinci argüman olarak bir string verin (isteğe bağlı olarak [sondaki virgülle](#)):

[VS Code'da aç](#) `cods/3_control_flow/3_error_handling/11_assert_with_message.dart`

```
assert(  
  urlString.startsWith('https'),  
  'URL ($urlString) should start with "https".',  
);
```

`assert` 'in ilk argümanı, boolean bir değer veren herhangi bir ifade olabilir. İfadenin değeri true ise doğrulama başarılı olur ve çalışma devam eder. false ise doğrulama başarısız olur ve bir istisna (bir [AssertionError](#)) fırlatılır.

Doğrulamalar tam olarak ne zaman çalışır? Bu, kullandığınız araçlara ve framework'e bağlıdır:

- Flutter doğrulamaları [debug modunda](#) etkinleştirir.
- [webdev serve](#) gibi yalnızca geliştirmeye yönelik araçlar genellikle doğrulamaları varsayılan olarak etkinleştirir.
- [dart run](#) ve [dart compile js](#) gibi bazı araçlar doğrulamaları bir komut satırı bayrağıyla destekler: `--enable-asserts`.

Üretim kodunda doğrulamalar yok sayılır ve `assert` 'e verilen argümanlar değerlendirilmez.

Fonksiyonlar

Dart'taki fonksiyonlar hakkında her şey.

Dart gerçek bir nesne yönelimli dildir; bu yüzden fonksiyonlar bile nesnedir ve bir tipleri vardır: **Function**. Bu, fonksiyonların değişkenlere atanabileceği ya da diğer fonksiyonlara argüman olarak geçirilebileceği anlamına gelir. Ayrıca bir Dart sınıfının örneğini bir fonksiyonmuş gibi çağırabilirsiniz. Ayrıntılar için [Callable objects](#) sayfasına bakın.

Bir fonksiyonun uygulanmasına örnek:

[VS Code'da aç](#) `cods/4_functions/1_functions/01_function.dart`

dart

```
bool isNoble(int atomicNumber) {
  return _nobleGases[atomicNumber] != null;
}
```

Effective Dart [public API'ler için tip belirtimi](#) önerse de tipleri yazmasanız da fonksiyon çalışır:

[VS Code'da aç](#) `cods/4_functions/1_functions/02_function_omitting_types.dart`

dart

```
isNoble(atomicNumber) {
  return _nobleGases[atomicNumber] != null;
}
```

Yalnızca tek bir ifade içeren fonksiyonlar için kısaltılmış bir sözdizimi kullanabilirsiniz:

[VS Code'da aç](#) `cods/4_functions/1_functions/03_function_shorthand.dart`

dart

```
bool isNoble(int atomicNumber) => _nobleGases[atomicNumber] != null;
```

`=>` *ifade* sözdizimi, `{ return ifade; }` ifadesinin kısaltmasıdır. `=>` gösterimine bazen *ok (arrow)* sözdizimi denir.

Not

Ok (`=>`) ile noktalı virgül (`;`) arasında yalnızca *ifadeler* bulunabilir. İfadeler bir değer üretir. Bu, Dart'ın bir değer beklediği yere bir deyim yazamayacağınız anlamına gelir. Örneğin bir [koşullu ifade](#) kullanabilirsiniz ama bir *if deyimi* kullanamazsınız. Önceki örnekte `_nobleGases[atomicNumber] != null;` boolean bir değer döndürür. Fonksiyon da `atomicNumber`'ın soy gaz aralığına girip girmediğini belirten boolean bir değer döndürür.

Parametreler

Bir fonksiyonun istediği sayıda *zorunlu konumsal* parametresi olabilir. Bunların ardından ya *isimli* parametreler ya da *isteğe bağlı konumsal* parametreler gelebilir (ikisi birden olamaz).

Not

Bazı API'ler, özellikle [Flutter](#) widget yapıcılar, zorunlu olan parametreler için bile yalnızca isimli parametreler kullanır. Ayrıntılar için sonraki bölüme bakın.

Bir fonksiyona argüman geçirirken ya da fonksiyon parametrelerini tanımlarken [sondaki virgülleri](#) kullanabilirsiniz.

İsimli parametreler

İsimli parametreler, açıkça `required` olarak işaretlenmedikçe isteğe bağlıdır.

Bir fonksiyon tanımlarken isimli parametreleri belirtmek için `{param1, param2, ...}` kullanın. Bir isimli parametreye varsayılan değer vermez ya da onu `required` olarak işaretlemeszeniz, varsayılan değeri `null` olacağı için tipi null olabilir (nullable) olmalıdır:

[VS Code'da aç](#) `cods/4_functions/1_functions/04_specify_named_parameters.dart`

dart

```
/// [bold] ve [hidden] bayraklarını ayarlar ...  
void enableFlags({bool? bold, bool? hidden}) {  
  ...  
}
```

Bir fonksiyonu çağırırken isimli argümanları `paramAdi: deger` şeklinde belirtebilirsiniz. Örneğin:

[VS Code'da aç](#) `cods/4_functions/1_functions/05_use_named_parameters.dart`

dart

```
enableFlags(bold: true, hidden: false);
```

Bir isimli parametre için `null` dışında bir varsayılan değer tanımlamak için `=` ile varsayılan değeri belirtin. Belirtilen değer bir derleme zamanı sabiti olmalıdır. Örneğin:

[VS Code'da aç](#) `cods/4_functions/1_functions/06_named_parameter_default_values.dart`

dart

```
/// [bold] ve [hidden] bayraklarını ayarlar ...  
void enableFlags({bool bold = false, bool hidden = false}) {  
  ...  
}  
  
// bold true olur; hidden false olur.  
enableFlags(bold: true);
```

Bunun yerine bir isimli parametrenin zorunlu olmasını, yani çağırانların parametre için bir değer vermesini istiyorsanız onu `required` ile işaretleyin:

[VS Code'da aç](#) `cods/4_functions/1_functions/07_required_named_parameters.dart`

dart

```
const Scrollbar({super.key, required Widget child});
```

Biri `child` argümanını belirtmeden bir `Scrollbar` oluşturmaya çalışırsa analizci bir sorun bildirir.

Not

`required` olarak işaretlenmiş bir parametre yine de null olabilir:

[VS Code'da aç](#) `cods/4_functions/1_functions/08_required_named_parameters_nullable.dart`

dart

```
const Scrollbar({super.key, required Widget? child});
```

Konumsal argümanları başa koymak isteyebilirsiniz, ama Dart bunu zorunlu tutmaz. Dart, API'nize uygun düştüğünde isimli argümanların argüman listesinin herhangi bir yerine konmasına izin verir:

[VS Code'da aç](#) `cods/4_functions/1_functions/09_named_arguments_anywhere.dart`

dart

```
repeat(times: 2, () {  
  ...  
});
```

İsteğe bağlı konumsal parametreler

Bir grup fonksiyon parametresini `[]` içine almak onları isteğe bağlı konumsal parametreler olarak işaretler. Varsayılan değer vermezseniz varsayılan değerleri `null` olacağı için tipleri `null` olabilir olmalıdır:

[VS Code'da aç](#) `cods/4_functions/1_functions/10_optional_positional_parameters.dart`

```
String say(String from, String msg, [String? device]) {
  var result = '$from says $msg';
  if (device != null) {
    result = '$result with a $device';
  }
  return result;
}
```

Bu fonksiyonu isteğe bağlı parametre olmadan çağırma örneği:

[VS Code'da aç](#) `cods/4_functions/1_functions/11_call_without_optional_param.dart`

```
assert(say('Bob', 'Howdy') == 'Bob says Howdy');
```

Bu fonksiyonu üçüncü parametreyle çağırma örneği ise şöyle:

[VS Code'da aç](#) `cods/4_functions/1_functions/12_call_with_optional_param.dart`

```
assert(
  say('Bob', 'Howdy', 'smoke signal') ==
    'Bob says Howdy with a smoke signal',
);
```

İsteğe bağlı bir konumsal parametre için `null` dışında bir varsayılan değer tanımlamak için `=` ile varsayılan değeri belirtin. Belirtilen değer bir derleme zamanı sabiti olmalıdır. Örneğin:

[VS Code'da aç](#) `cods/4_functions/1_functions/13_optional_positional_param_default.dart`

```
String say(String from, String msg, [String device = 'carrier pigeon']) {
  var result = '$from says $msg with a $device';
  return result;
}

assert(say('Bob', 'Howdy') == 'Bob says Howdy with a carrier pigeon');
```

Parametre niteleyicileri

`final` ve `var` parametre niteleyicileri yalnızca örnek alanları tanımlamak için **birincil yapıcılara (primary constructor)** ayrılmıştır. Bunları başka herhangi bir tanımdaki biçimsel parametrelerde kullanmak derleme zamanı hatası (`extraneous_modifier`) üretir. Kod örnekleri ve geçiş adımları için [Parameter modifier restrictions](#) bölümüne bakın.

Bir stil tercihi olarak parametrelere yeniden değer atanamamasını zorunlu kılmak için bunun yerine [parameter_assignments](#) lint kuralını kullanın.

main() fonksiyonu

Her uygulamanın, uygulamanın giriş noktası işlevi gören üst düzey bir `main()` fonksiyonu olmalıdır. `main()` fonksiyonu `void` döndürür ve argümanlar için isteğe bağlı bir `List<String>` parametresi vardır.

İşte basit bir `main()` fonksiyonu:

[VS Code'da aç](#) `cods/4_functions/1_functions/14_hello_world.dart`

```
void main() {
  print('Hello, World!');
}
```

Argüman alan bir komut satırı uygulaması için `main()` fonksiyonu örneği:

[VS Code'da aç](#) `cods/4_functions/1_functions/15_main_args.dart`

args.dart

dart

```
// Uygulamayı şöyle çalıştırın: dart run args.dart 1 test
void main(List<String> arguments) {
  print(arguments);

  assert(arguments.length == 2);
  assert(int.parse(arguments[0]) == 1);
  assert(arguments[1] == 'test');
}
```

Komut satırı argümanlarını tanımlamak ve ayırtmak için [args kütüphanesini](#) kullanabilirsiniz.

Birinci sınıf nesneler olarak fonksiyonlar

Bir fonksiyonu başka bir fonksiyona parametre olarak geçirebilirsiniz. Örneğin:

[VS Code'da aç](#) `cods/4_functions/1_functions/16_function_as_param.dart`

dart

```
void printElement(int element) {
  print(element);
}

var list = [1, 2, 3];

// printElement'i parametre olarak geçir.
list.forEach(printElement);
```

Bir fonksiyonu bir değişkene de atayabilirsiniz, örneğin:

[VS Code'da aç](#) `cods/4_functions/1_functions/17_function_as_var.dart`

dart

```
var loudify = (msg) => '!!! ${msg.toUpperCase()} !!!';
assert(loudify('hello') == '!!! HELLO !!!');
```

Bu örnekte anonim bir fonksiyon kullanılıyor. Bunlar hakkında daha fazlası sonraki bölümde.

Fonksiyon tipleri

Bir fonksiyonun tipini belirtebilirsiniz; buna *fonksiyon tipi* denir. Fonksiyon tipi, bir fonksiyon tanımının başlığında fonksiyon adının yerine `Function` anahtar kelimesi konarak elde edilir. Ayrıca konumsal parametrelerin adlarını yazmayabilirsiniz, ancak isimli parametrelerin adları atlanamaz. Örneğin:

[VS Code'da aç](#) `cods/4_functions/1_functions/18_function_type.dart`

dart

```
void greet(String name, {String greeting = 'Hello'}) =>
  print('$greeting $name!');

// 'greet'i bir değişkende sakla ve çağır.
void Function(String, {String greeting}) g = greet;
g('Dash', greeting: 'Howdy');
```

Not

Dart'ta fonksiyonlar birinci sınıf nesnelerdir; yani değişkenlere atanabilir, argüman olarak geçirilebilir ve başka fonksiyonlardan döndürülebilirler.

Fonksiyon tiplerine açıkça isim vermek için bir `typedef` tanımı kullanabilirsiniz; bu, açıklık ve yeniden kullanılabilirlik açısından faydalı olabilir.

Anonim fonksiyonlar

`main()` ya da `printElement()` gibi çoğu fonksiyona isim verseniz de isimsiz fonksiyonlar da oluşturabilirsiniz. Bu fonksiyonlara *anonim fonksiyon*, *lambda* ya da *closure* denir.

Anonim bir fonksiyon isimli bir fonksiyona benzer, çünkü şunlara sahiptir:

- Virgülle ayrılmış sıfır ya da daha fazla parametre
- Parantez içinde isteğe bağlı tip belirtimleri.

Aşağıdaki kod bloğu fonksiyonun gövdesini içerir:

[VS Code'da aç](#) `cods/4_functions/1_functions/19_ornek.dart`

dart

```
([[Type] param1[, ...]]) {  
  codeBlock;  
}
```

Aşağıdaki örnek, tipi belirtilmemiş `item` parametresine sahip anonim bir fonksiyonu `map` fonksiyonuna geçirir. Listedeki her eleman için çağrılan `map` fonksiyonu her string'i büyük harfe çevirir. Ardından bir `for-in` döngüsü dönüştürülen her string'i uzunluğuyla birlikte yazdırır.

[VS Code'da aç](#) `cods/4_functions/1_functions/20_anonymous_function.dart`

dart

```
const list = ['apples', 'bananas', 'oranges'];  
  
var uppercaseList = list.map((item) {  
  return item.toUpperCase();  
}).toList();  
// map işleminden sonra listeye çevir  
  
for (var item in uppercaseList) {  
  print('$item: ${item.length}');  
}
```

Bu örneği [DartPad](#)'de çalıştırabilirsiniz.

Fonksiyon yalnızca tek bir ifade ya da return deyimi içeriyorsa onu ok gösterimiyle kısaltabilirsiniz. İşlevsel olarak aynı olduğunu doğrulamak için aşağıdaki satırı DartPad'e yapıştırıp **Run**'a tıklayın.

[VS Code'da aç](#) `cods/4_functions/1_functions/21_anon_func.dart`

dart

```
var uppercaseList = list.map((item) => item.toUpperCase()).toList();
```

Sözcüksel kapsam (lexical scope)

Dart değişkenlerin kapsamını kodun yerleşimine göre belirler. Bu özelliğe sahip bir programlama diline sözcüksel kapsamlı dil denir. Bir değişkenin kapsam içinde olup olmadığını görmek için "süslü parantezleri dışarı doğru takip edebilirsiniz".

Örnek: Her kapsam düzeyinde değişkenleri olan bir dizi iç içe fonksiyon:

[VS Code'da aç](#) cods/4_functions/1_functions/22_nested_functions.dart

dart

```
bool topLevel = true;

void main() {
  var insideMain = true;

  void myFunction() {
    var insideFunction = true;

    void nestedFunction() {
      var insideNestedFunction = true;

      assert(topLevel);
      assert(insideMain);
      assert(insideFunction);
      assert(insideNestedFunction);
    }
  }
}
```

`nestedFunction()` metodu, en üst düzeye kadar her düzeydeki değişkenleri kullanabilir.

Sözcüksel closure'lar

Kendi sözcüksel kapsamının dışında bulunduğu bile o kapsamdaki değişkenlere erişebilen fonksiyon nesnesine *closure* denir.

Fonksiyonlar, çevreleyen kapsamlarda tanımlanmış değişkenleri yakalayabilir (close over). Aşağıdaki örnekte `makeAdder()`, `addBy` değişkenini yakalar. Döndürülen fonksiyon nereye giderse gitsin `addBy`'ı hatırlar.

[VS Code'da aç](#) cods/4_functions/1_functions/23_function_closure.dart

dart

```
/// Fonksiyonun argümanına [addBy] ekleyen
/// bir fonksiyon döndürür.
Function makeAdder(int addBy) {
  return (int i) => addBy + i;
}

void main() {
  // 2 ekleyen bir fonksiyon oluştur.
  var add2 = makeAdder(2);

  // 4 ekleyen bir fonksiyon oluştur.
  var add4 = makeAdder(4);

  assert(add2(3) == 5);
  assert(add4(3) == 7);
}
```

Tear-off'lar

Bir fonksiyona, metoda ya da isimli yapıcuya parantezsiz başvurduğunuzda Dart bir *tear-off* oluşturur. Bu, fonksiyonla aynı parametreleri alan ve çağırıldığında asıl fonksiyonu çağırان bir closure'dır. Kodunuz, closure'ın aldığı parametrelerle isimli bir fonksiyonu çağırان bir closure'a ihtiyaç duyuyorsa çağırayı bir lambda'ya sarmayın. Bir tear-off kullanın.

[VS Code'da aç](#) cods/4_functions/1_functions/24_variables.dart

dart

```
var charCodes = [68, 97, 114, 116];
var buffer = StringBuffer();
```

[VS Code'da aç](#) cods/4_functions/1_functions/25_good_example.dart

iyi

dart

```
// Fonksiyon tear-off'u
charCodes.forEach(print);

// Metot tear-off'u
charCodes.forEach(buffer.write);
```

[VS Code'da aç](#) cods/4_functions/1_functions/26_bad_example.dart

kötü

dart

```
// Fonksiyon lambdası
charCodes.forEach((code) {
  print(code);
});

// Metot lambdası
charCodes.forEach((code) {
  buffer.write(code);
});
```

Fonksiyonların eşitliğini test etme

Üst düzey fonksiyonların, statik metotların ve örnek metotlarının eşitliğini test etmeye bir örnek:

[VS Code'da aç](#) cods/4_functions/1_functions/27_ornek.dart

dart

```
void foo() {} // Üst düzey bir fonksiyon

class A {
  static void bar() {} // Statik bir metot
  void baz() {} // Bir örnek (instance) metodu
}

void main() {
  Function x;

  // Üst düzey fonksiyonları karşılaştırma.
  x = foo;
  assert(foo == x);

  // Statik metotları karşılaştırma.
  x = A.bar;
  assert(A.bar == x);

  // Örnek metotlarını karşılaştırma.
  var v = A(); // A'nın 1. örneği
  var w = A(); // A'nın 2. örneği
  var y = w;
  x = w.baz;

  // Bu closure'lar aynı örneğe (#2) başvurur,
  // bu yüzden eşittirler.
  assert(y.baz == x);

  // Bu closure'lar farklı örneklere başvurur,
  // bu yüzden eşit değildirler.
  assert(v.baz != w.baz);
}
```

Dönüş değerleri

Tüm fonksiyonlar bir değer döndürür. Dönüş değeri belirtilmemişse fonksiyon gövdesinin sonuna örtük olarak `return null;` deyimi eklenir.

[VS Code'da aç](#) `cods/4_functions/1_functions/28_implicit_return_null.dart`

dart

```
foo() {}

assert(foo() == null);
```

Bir fonksiyonda birden çok değer döndürmek için değerleri bir [record](#) içinde toplayın.

[VS Code'da aç](#) `cods/4_functions/1_functions/29_ornek.dart`

dart

```
(String, int) foo() {
  return ('something', 42);
}
```

Getter'lar ve setter'lar

Her özellik erişimi (üst düzey, statik ya da örnek) bir getter'ın ya da setter'ın çağırılmasıdır. Bir değişken örtük olarak bir getter, değiştirilebilirse bir de setter oluşturur. Bu yüzden bir özelliğe eriştiğinizde aslında arka planda küçük bir fonksiyon çağırırsınız. Bir özelliği okumak bir getter fonksiyonunu, yazmak ise bir setter fonksiyonunu çağırır; özellik bir değişken olarak tanımlanmış olsa bile.

Bununla birlikte getter ve setter'ları sırasıyla `get` ve `set` anahtar kelimeleriyle açıkça da tanımlayabilirsiniz. Bu, bir özelliğin değerinin okunurken ya da yazılırken hesaplanmasını sağlar.

Getter ve setter kullanmanın amacı, istemci (özellik kullanan kod) ile sağlayıcı (onu tanımlayan sınıf ya da kütüphane) arasında net bir ayrım oluşturmaktır. İstemci, değer basit bir değişkende mi saklandığını yoksa o anda mı hesaplandığını bilmeye gerek kalmadan bir değeri ister ya da ayarlar. Bu da sağlayıcıya özelliğin çalışma şeklini değiştirme özgürlüğü verir.

Örneğin özelliğin değeri hiçbir yerde saklanmıyor olabileceği için getter her çağrıldığında yeniden hesaplanabilir. Bir başka örnek de değerin private bir değişkende saklandığı ve public erişime yalnızca bir getter ya da setter çağrılarak izin verildiği durumdur.

Aşağıdaki örnek bunu gösteriyor: `secret` getter'ı ve setter'ı, atanan ve okunan değerler üzerinde kendi işlemlerini yaparak private `_secret` değişkenine dolaylı erişim sağlar.

VS Code'da aç cods/4_functions/1_functions/30_ornek.dart

dart

```
// Tanımlayıcısı alt çizgiyle ('_') başladığı için kütüphaneye özel olan
// '_secret' adlı bir değişken tanımlar.
String _secret = 'Hello';

// [_secret]'a okuma erişimi sağlayan
// public, üst düzey bir getter.
String get secret {
  print('Getter was used!');
  return _secret.toUpperCase();
}

// [_secret]'a yazma erişimi sağlayan
// public, üst düzey bir setter.
set secret(String newMessage) {
  print('Setter was used!');
  if (newMessage.isNotEmpty) {
    _secret = newMessage;
    print('New secret: "$newMessage"');
  }
}

void main() {
  // Değeri okumak getter'ı çağırır.
  print('Current message: $secret');

  /*
  Çıktı:
  Getter was used!
  Current message: HELLO
  */

  // Değer atamak setter'ı çağırır.
  secret = 'Dart is fun';

  // Tekrar okumak, yeni hesaplanan değeri göstermek için getter'ı çağırır
  print('New message: $secret');

  /*
  Çıktı:
  Setter was used! New secret: "Dart is fun"
  Getter was used!
  New message: DART IS FUN
  */
}
```

Üreteçler (generators)

Bir değer dizisini tembel (lazy) şekilde üretmeniz gerektiğinde bir *üreteç fonksiyonu* kullanmayı düşünün. Dart'ta iki tür üreteç fonksiyonu için yerleşik destek vardır:

- **Senkron** üreteç: Bir `Iterable` nesnesi döndürür.
- **Asenkron** üreteç: Bir `Stream` nesnesi döndürür.

Senkron bir üreteç fonksiyonu yazmak için fonksiyon gövdesini `sync*` olarak işaretleyin ve değerleri vermek için `yield` deyimlerini kullanın:

[VS Code'da aç](#) `cods/4_functions/1_functions/31_sync_generator.dart`

dart

```
Iterable<int> naturalsTo(int n) sync* {  
  int k = 0;  
  while (k < n) yield k++;  
}
```

Asenkron bir üreteç fonksiyonu yazmak için fonksiyon gövdesini `async*` olarak işaretleyin ve değerleri vermek için `yield` deyimlerini kullanın:

[VS Code'da aç](#) `cods/4_functions/1_functions/32_async_generator.dart`

dart

```
Stream<int> asynchronousNaturalsTo(int n) async* {  
  int k = 0;  
  while (k < n) yield k++;  
}
```

Üreticiniz özyinelemeliyse `yield*` kullanarak performansını artırabilirsiniz:

[VS Code'da aç](#) `cods/4_functions/1_functions/33_recursive_generator.dart`

dart

```
Iterable<int> naturalsDownFrom(int n) sync* {  
  if (n > 0) {  
    yield n;  
    yield* naturalsDownFrom(n - 1);  
  }  
}
```

Harici (external) fonksiyonlar

Harici fonksiyon, gövdesi tanımından ayrı olarak uygulanan bir fonksiyondur. Bir fonksiyon tanımının önüne şu şekilde `external` anahtar kelimesini ekleyin:

[VS Code'da aç](#) `cods/4_functions/1_functions/34_ornek.dart`

dart

```
external void someFunc(int i);
```

Harici bir fonksiyonun uygulaması başka bir Dart kütüphanesinden ya da daha yaygın olarak başka bir dilden gelebilir. Birlikte çalışma (interop) bağlamlarında `external`, yabancı fonksiyonlar ya da değerler için tip bilgisi sunarak onları Dart'ta kullanılabilir hale getirir. Uygulama ve kullanım büyük ölçüde platforma özgüdür; daha fazla bilgi için örneğin [C](#) ya da [JavaScript](#) interop dokümantasyonuna göz atın.

Harici fonksiyonlar üst düzey fonksiyonlar, [örnek metotları](#), getter ya da setter'lar veya [yönlendirmeyen yapıcılar](#) olabilir. Bir [örnek değişkeni](#) de `external` olabilir; bu, harici bir getter'a ve (değişken `final` değilse) harici bir setter'a denktir.